

4. Ingeniería de Datos

[Redactar una breve introducción al capítulo de forma general y al final particularizando para las secciones en las que se divide].

4.1. Infraestructura de Desarrollo

La fase inicial del diseño e ingeniería de datos se fundamenta en la creación de un entorno de experimentación controlado que garantiza la reproductibilidad de los resultados y la eficiencia en el flujo de trabajo de la Inteligencia Artificial (IA). Para ello, se ha estructurado un ecosistema que integra hardware de alto rendimiento para el entrenamiento de modelos y una organización modular de archivos. El desarrollo, entrenamiento y validación de las arquitecturas de aprendizaje profundo se realizaron mediante una infraestructura híbrida conformada por dos estaciones de trabajo:

- **Estación de trabajo local:** Se utiliza una unidad de procesamiento equipada con un chip Apple M1 (arquitectura *Advanced RISC Machine* [ARM] de 8 núcleos) y 8GB de memoria de acceso aleatorio (*Random Access Memory* [RAM]) unificada. Este entorno constituye el núcleo de desarrollo primario, donde se gestiona la lógica de programación mediante entornos virtuales de Python y control de versiones. La eficiencia de su arquitectura permite ejecutar el procesamiento de datos (Fase 1), la validación de módulos y pruebas de inferencia a escala reducida (Fase 2), además de la configuración del microcontrolador ESP32-S3 mediante el entorno *ESP-IDF*. Funciona como un laboratorio de prototipado rápido para asegurar la corrección sintáctica de los procesos (pipelines) de entrenamiento antes de su escalamiento hacia la infraestructura en la nube.
- **Cloud-native o MLOps:** Implementa el paradigma de operaciones de aprendizaje automático (*Machine Learning Operations* [MLOps]) a través de *Vertex AI Custom Training*. El ecosistema consiste en un nodo de entrenamiento configurado como un trabajo personalizado (*Custom Job*), el cual despliega una máquina virtual *n1-standard-8* (8 vCPUs y 30 GB de RAM) potenciada por un acelerador de hardware *NVIDIA Tesla T4*. Se selecciona este acelerador por su equilibrio entre capacidad de cómputo paralelo y eficiencia frente a otras soluciones de mayor escala. La arquitectura se

apoya en un repositorio centralizado de *Google Cloud Storage (GCS)* para la persistencia de los conjuntos de datos (*datasets*) y la exportación de los artefactos de los modelos resultantes en cada ciclo de experimentación. La orquestación se realiza de forma programática mediante el Kit de Desarrollo de Software (*Software Development Kit [SDK]*) de *Google Cloud Platform (GCP)*.

La elección de esta infraestructura híbrida responde a la necesidad de equilibrar la agilidad en el desarrollo de software con la potencia de cómputo requerida por los algoritmos de visión artificial. Esta configuración garantiza eficiencia en el ciclo de desarrollo, escalabilidad y una trazabilidad rigurosa en la gestión de datos.

4.2. Gestión de software y estructura del proyecto

Para la gestión del software y sus dependencias en la estación de trabajo local, se ha implementado un entorno virtual aislado mediante el gestor de paquetes Conda, utilizando la versión 3.10 del lenguaje de programación Python. Esta configuración garantiza la estabilidad de las librerías críticas y previene conflictos técnicos entre las herramientas de análisis de datos y los motores de inferencia. Dentro de este ecosistema, se integran bibliotecas fundamentales categorizadas de la siguiente manera:

- **Procesamiento y visión artificial:** Se emplean *OpenCV-Python* y *Scikit-Image* para la manipulación avanzada de imágenes y la visualización de cajas delimitadoras. Por su parte, *PyCOCOTools* sirve como interfaz estándar para el manejo de anotaciones en formato *Common Objects in Context (COCO)*.
- **Análisis y estructura de datos:** Las librerías *Pandas*, *NumPy*, *Matplotlib* y *Seaborn* constituyen el núcleo de herramientas para el Análisis Exploratorio de Datos (EDA, por sus siglas en inglés) y la generación de reportes métricos.
- **Eficiencia en el desarrollo:** Se utiliza *IPyKernel* para integrar el entorno virtual en Visual Studio Code como entorno de desarrollo integrado (IDE) principal. Esto permite una ejecución segmentada mediante cuadernos de *Jupyter* y el control de calidad del código a través de la herramienta de análisis estático *Pylint*.

La organización del proyecto adopta un esquema de ingeniería de software orientado a la separación de responsabilidades entre la lógica de datos, el desarrollo de modelos y el despliegue en el microcontrolador ESP32-S3. Además, no solo mejora la mantenibilidad del proyecto, sino que también facilita la auditoría técnica de los resultados obtenidos en cada fase de la investigación. La estructura modular se divide en tres módulos principales:

- **01_ING_DATOS:** Dedicado exclusivamente al almacenamiento, curación y persistencia del conjunto de datos (dataset).
- **02_ING_MODELOS:** Contiene los guiones (*scripts*) y módulos de la lógica de entrenamiento, el sistema de registro (logging) para la trazabilidad de experimentos y exportación a *Vertex AI Custom Training*.
- **03_DESPLIEGUE_ESP32S3:** Integra la configuración bajo la extensión ESP-IDF necesaria para la implementación de los modelos en el hardware ESP32-S3 WROOM N16R8.

4.3. Especificaciones del Hardware del ESP32-S3

La implementación de modelos de aprendizaje profundo en sistemas embebidos requiere una plataforma que equilibre la eficiencia energética con capacidades de cómputo y memoria superiores a las de los microcontroladores de propósito general. Por esta razón, se ha seleccionado el módulo ESP32-S3-WROOM-1 (N16R8), ver Figura 2, como el núcleo de procesamiento para esta investigación.

La Tabla 6 muestra la comparativa frente a otras plataformas disponibles en el mercado, con características probadas en TinyML y los motivos por los que destaca la ESP32-S3.

Figura 2. Kit Freenove® ESP32-S3 para aprendizaje de programación y electrónica



Fuente: Freenove (2025)

Tabla 6. Comparativa de ESP32-S3 frente a otras plataformas de hardware

Característica	ESP32-S3 (N16R8)	ESP32 (Original)	STM32H7 (High-End)	K210 (RISC-V)	Justificación para S3
Núcleo	Dual Xtensa LX7	Dual Xtensa LX6	ARM Cortex-M7	Dual RISC-V 64-bit	S3 ofrece instrucciones vectoriales ausentes en ESP32 original; costo menor que STM32H7.
Aceleración AI	Extensiones Vectoriales (SIMD)	Ninguna	Unidad MAC / Chrom-ART	KPU (Acelerador CNN dedicado)	K210 tiene KPU pero un toolchain rígido y obsoleto. S3 ofrece flexibilidad de software moderna (TFLite/ESP-DL).
Memoria RAM	8MB PSRAM (Octal)	4MB PSRAM (Quad)	Variable (usualmente ext.)	8MB SRAM (rápida)	La PSRAM Octal del S3 ofrece un balance ideal de costo-rendimiento frente a la SRAM costosa del K210.
Conectividad	Wi-Fi + BLE 5.0	Wi-Fi + BT 4.2	Generalmente externa	Ninguna (usualmente)	S3 integra radio, esencial para Edge AI (envío de alertas/telemetría) sin chips extra.
Ecosistema	ESP-IDF, Arduino, PlatformIO	Maduro	Maduro (CubeAI)	Fragmentado (Kendryte)	Soporte masivo de Espressif y comunidad para librerías de IA modernas.

4.3.1. Arquitectura del SoC: Aceleración Vectorial y Núcleo Xtensa LX7

El microcontrolador ESP32-S3 está impulsado por un procesador de doble núcleo Xtensa® LX7 de 32 bits, capaz de operar a una frecuencia de hasta 240 MHz. A diferencia de sus predecesores, el núcleo LX7 introduce una tubería de ejecución de 7 etapas y, fundamentalmente, la inclusión de instrucciones vectoriales (SIMD - *Single Instruction, Multiple Data*).

Esta característica es el principal habilitador de Edge AI en este chip, ya que permite realizar múltiples operaciones de Multiplicación y Acumulación (MAC) en un solo ciclo de reloj. El uso de estas instrucciones, orquestadas a través de bibliotecas optimizadas como ESP-DL o ESP-NN, permite reducir la latencia de inferencia en factores de 3x a 5x en comparación con implementaciones escalares, transformando el SoC en una unidad de procesamiento neuronal definida por software (Espressif Systems, 2026).

4.3.2. Jerarquía de Memoria N16R8

La variante seleccionada (N16R8) es fundamental para superar los cuellos de botella de memoria inherentes a la visión artificial, en especial en los sistemas embebidos para visión artificial, donde la memoria es el recurso más restrictivo, superando incluso a la potencia de cómputo. El sufijo “N16R8” en el módulo seleccionado, denota las siguientes características:

- **Almacenamiento Flash de 16 MB (N16):** Esta capacidad permite alojar archivos binarios de modelos de detección de objetos que, tras la cuantización, pueden alcanzar tamaños de entre 2 y 6 MB, dependiendo de sus hiperparámetros, arquitectura y la resolución de entrada. Otros microcontroladores cuentan con límites de hasta 4MB de memoria flash que los hacen inviables para modelos de alta eficacia, sacrificando la precisión o la funcionalidad de la aplicación. La capacidad de 16 MB del N16R8 permite almacenar modelos complejos, mantener múltiples versiones de modelos para pruebas A/B en campo, y reservar espacio suficiente para buffers de registro, particiones de actualización inalámbrica (OTA) y sistemas de archivo como SPIFFS/LittleFS. (Espressif Systems, 2025).

- **PSRAM Octal de 8 MB (R8):** Dado que la SRAM interna de 512 KB es insuficiente para gestionar el *Tensor Arena* (espacio de trabajo de inferencia) y los búferes de imagen simultáneamente, el uso de PSRAM externa es obligatorio. La interfaz Octal SPI (OPI) duplica el ancho de banda efectivo en comparación con las interfaces Quad SPI tradicionales, permitiendo el manejo de tensores intermedios y mapas de activación de modelos de Deep Learning sin degradar severamente el rendimiento por latencia de acceso. Esto mitiga significativamente la penalización de latencia asociada con el acceso a la RAM externa, permitiendo que el ESP32-S3 manipule tensores de entrada de resoluciones como 224x224 o 320x320 RGB, que son estándares para modelos de transferencia de aprendizaje.

4.3.3. Subsistema de Visión: Sensor OV5640

Para la adquisición de imágenes se utiliza el sensor OmniVision OV5640 de 5 megapíxeles, el cual es tácticamente vital frente a versiones alternativas como OV2640, por dos razones importantes (OmniVision Technologies, 2011):

- **Procesamiento de Región de Interés (ROI):** La alta resolución del sensor permite implementar pipelines de *Zoom Digital sin Pérdida*, manejar diferentes formatos y resoluciones de imágenes (Ver Tabla 7). El sistema puede capturar una imagen de alta resolución, identificar una sub-región específica, recortar esa área y luego alimentar solo ese recorte a la red neuronal. Esto permite inferencia de alta precisión a distancia, algo imposible con sensores de baja resolución nativa.
- **Enfoque Automático (AF):** Muchos módulos OV5640 incluyen capacidades de enfoque automático. Para aplicaciones variables donde la distancia al objeto no es fija, el enfoque óptico es superior al enfoque fijo, garantizando que las características de textura fina necesarias para la convolución se preserven.

Tabla 7. Resoluciones Estándar de Sensor OV5640

Constante en Código (framesize_t)	Resolución (Píxeles)	Nombre Común
FRAMESIZE_QQVGA	160 x 120	QQVGA
FRAMESIZE_QCIF	176 x 144	QCIF
FRAMESIZE_HQVGA	240 x 176	HQVGA
FRAMESIZE_QVGA	320 x 240	QVGA
FRAMESIZE_CIF	400 x 296	CIF
FRAMESIZE_VGA	640 x 480	VGA
FRAMESIZE_SVGA	800 x 600	SVGA
FRAMESIZE_XGA	1024 x 768	XGA
FRAMESIZE_SXGA	1280 x 1024	SXGA
FRAMESIZE_UXGA	1600 x 1200	UXGA (2MP)
FRAMESIZE_QXGA	2048 x 1536	QXGA (3MP)
FRAMESIZE_QSXGA	2592 x 1944	5 Megapíxeles

Fuente: Elaboración propia a partir de datos de OmniVision Technologies (2011)

4.3.4. Requisitos Técnicos Objetivo para Deep Learning en ESP32-S3

Para ampliar la justificación de la elección de la ESP32-S3 como hardware para esta investigación, se presenta, a continuación, la Tabla 8 con los requisitos técnicos objetivo (*target specs*) obtenidos a partir de la revisión documental del estado del arte realizada en la preparación de esta etapa de la investigación.

Tabla 8. Requisitos Técnicos Objetivo para Deep Learning en ESP32-S3

Métrica	Objetivo Óptimo	Límite Estricto	Justificación Técnica
Peso del Modelo (Flash)	< 2.5 MB	< 10 MB	Aunque el N16 tiene 16MB, el código de la aplicación, las particiones OTA, y el sistema de archivos (SPIFFS/LittleFS) requieren espacio. Cargar pesos grandes desde Flash a RAM induce latencia.
Consumo de RAM (Arena)	< 2 MB	< 4 MB	El S3 tiene 8MB PSRAM, pero asignar más de 4MB al <i>Tensor Arena</i> deja poco espacio para los buffers de imagen de la cámara y los stacks de Wi-Fi/BLE.
Resolución de Entrada	96x96 a 224x224	640x640	El costo computacional crece cuadráticamente con la resolución. Procesar >640x640 genera una explosión en operaciones MAC que el S3 no puede manejar en tiempo real.
Cuantización	INT8	INT8	Obligatorio. FP32 (punto flotante) es demasiado lento y consume 4x memoria. Las instrucciones vectoriales del S3 están optimizadas específicamente para aritmética INT8/INT16.
Latencia de Inferencia	< 200 ms	< 800 ms	Para monitoreo estático, hasta 800ms es aceptable. Para monitoreo dinámico, hasta 500 ms es aceptable.

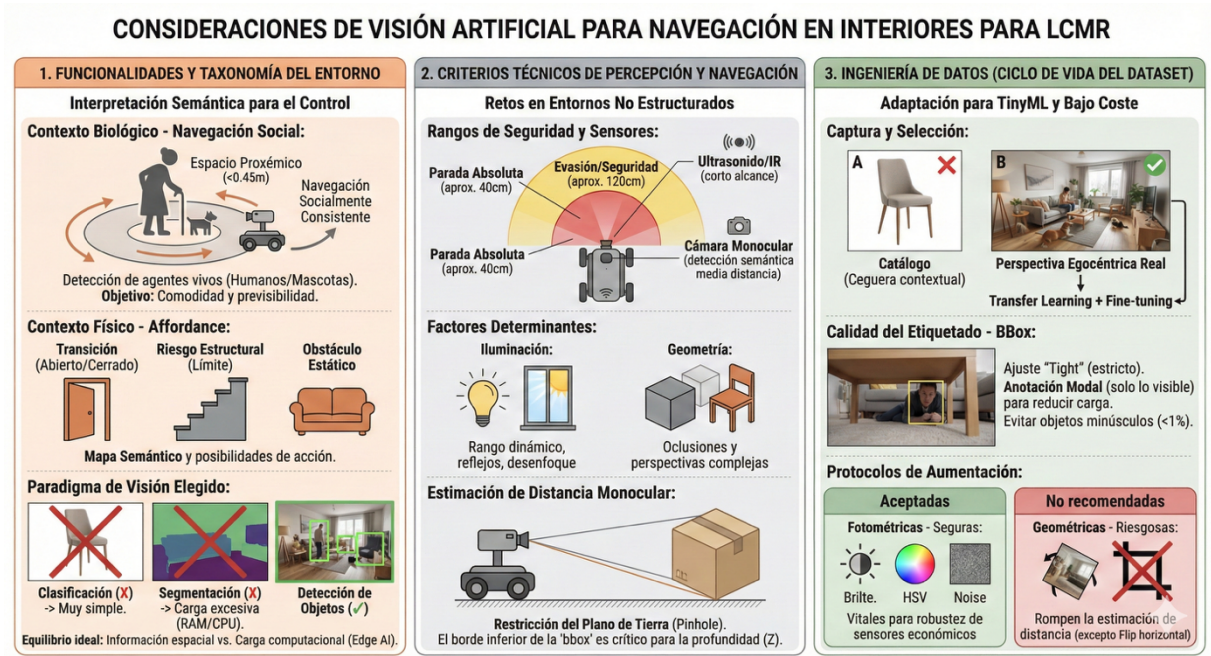
Métrica	Objetivo Óptimo	Límite Estricto	Justificación Técnica
Número de Parámetros	< 1,000,000	< 2,000,000	Más parámetros implican más accesos a memoria para traer pesos. Mantener los modelos "delgados" es importante para evitar cuellos de botella en el bus de memoria.
Modo de Flash	QIO 80MHz	DIO 40MHz	La velocidad de lectura de la Flash impacta directamente en la carga de capas del modelo. Es vital configurar el modo QIO a la máxima frecuencia.
Velocidad de CPU	240 MHz	160 MHz	Potencia las instrucciones vectoriales, acelerando las operaciones matemáticas complejas, reduciendo la latencia de inferencia y gestión óptima de la PSRAM.

Fuente: Elaboración propia

4.4. Adquisición y Caracterización del dataset

La visión artificial aplicada en entornos reales de navegación en interiores para un LCMR supone, como fue discutido en las secciones: 2.2.4., 2.2.5 y 2.2.6 del estado del arte, diversas consideraciones a nivel de funcionalidades, criterios técnicos e ingeniería de datos, resumidos en la Figura 3.

Figura 3. Consideraciones de visión artificial para navegación en interiores con LCMR



Nota: Imagen generada mediante el modelo de inteligencia artificial Nano Banana Pro (Google, 2026) a partir del estado del arte argumentado en las secciones: 2.2.4., 2.2.5 y 2.2.6 de esta investigación.

A partir de los criterios establecidos, se fundamentó la recopilación del conjunto de datos (*dataset*) que integran las siguientes clases de objetos:

- Perros (*`dog`*): representación de mascotas dentro del contexto biológico.
- Personas (*`person`*): representación de seres humanos en el contexto biológico.
- Puertas (*`door`*): representación del *Affordance* o contexto físico de transición de espacios.
- Escaleras (*`stair`*): representación del *Affordance* o contexto físico de riesgo estructural o límite de actuación del LCMR.
- Obstáculos (*`obstacle`*): representación del *Affordance* o contexto físico de diversidad de objetos estáticos y diferentes.

Con la definición de estas clases, se alcanza una representatividad mínima que proporcione al LCMR la información necesaria para la navegación social y semántica. Esto facilitará la planificación, ejecución y corrección de misiones en entornos interiores.

Debido a que se trata de un entorno controlado y con el fin de implementar una estrategia de entrenamiento basada en *Transfer Learning* y *Fine Tuning* de modelos preentrenados, las imágenes recopiladas cumplen con los siguientes criterios técnicos:

- **Perspectivas egocéntricas:** inclusión de capturas reales y sintéticas; estas últimas se emplean para mitigar el overfitting.
- **Variabilidad de clases:** distribución en diversos escenarios con un máximo de 15 objetos por imagen.
- **Control de oclusión:** Prevenir la oclusión de objetos en una proporción superior al 30%.
- **Rangos de detección:** Identificación de objetos a distancias de entre 20 y 150 centímetros para evitar el etiquetado de elementos minúsculos o excesivamente grandes.
- **Calidad de imagen:** Captura en rangos dinámicos de iluminación, reflejos y desenfoques naturales del sensor utilizado y con resoluciones inferiores a 640x640 píxeles.
- **Multiplicidad de ángulos:** Registro de diferentes ángulos de los mismos objetos donde la cámara está ubicada a una altura/distancia de entre 10 y 40 cm del suelo, enfocando objetos con perspectivas de “gusano”.

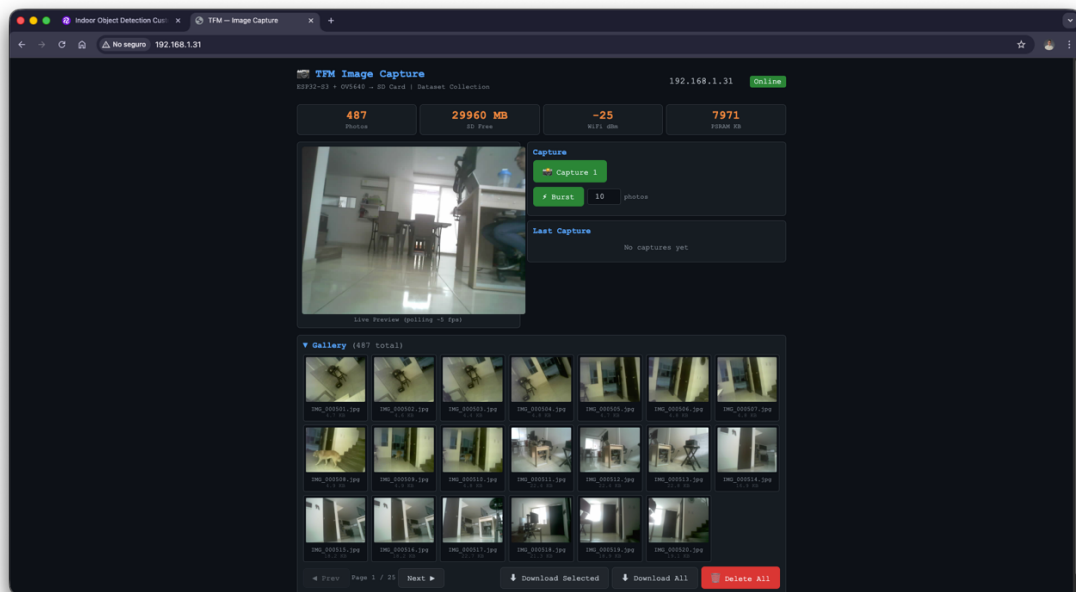
- **Restricción espacial:** Garantía de visibilidad del plano de tierra y del borde inferior de las cajas delimitadoras.
- **Etiquetado especializado:** Anotación mediante ajuste estricto (*tight*) y modal (recuadro limitado al objeto visible).

El dataset final se compone de dos diferentes fuentes:

4.4.1. Dataset Egocéntrico Real

Estas imágenes fueron capturadas directamente desde la plataforma ESP32-S3 WROOM N16R8 + CAM OV5640 a través de una interfaz web diseñada (ver Figura 4) específicamente para la captura, recopilación y descarga de imágenes. El entorno de navegación real propuesto como laboratorio de pruebas de esta investigación tiene un área de aproximadamente treinta metros cuadrados ($30m^2$). El entorno se caracteriza por ser doméstico interior (*indoor home*) y combinar diversidad en las cinco clases de objetos previamente definidas.

Figura 4. Interfaz Web para captura de Dataset Egocéntrico Real



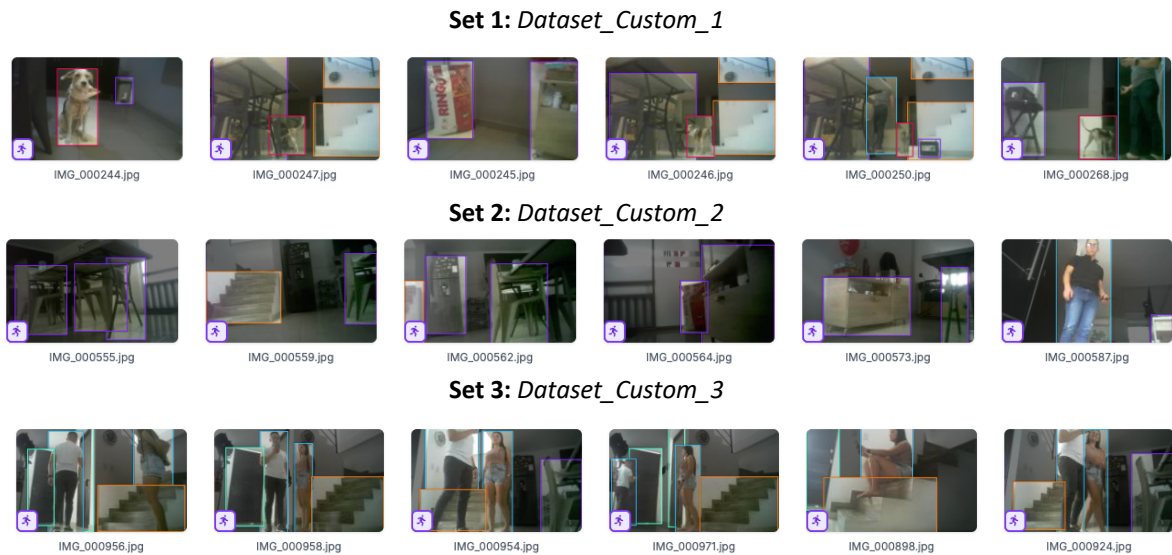
Nota: Elaboración propia desarrollada en estación de trabajo local – VSCode + extensión ESP-IDF y Web Socket en ESP32-S3+OV5640+SDCARD (C++, HTML, CSS, JS)

En la figura 5 se observan algunas muestras de imágenes recopiladas desde esta fuente, las cuales fueron obtenidas en tres *sets* o subconjuntos de imágenes:

- **Dataset_Custom_1:** 512 Fotos tomadas a resolución FRAMSIZE_QVGA 320x240 pixeles [baja iluminación - noche]

- **Dataset_Custom_2:** 265 Fotos tomadas a resolución FRAMSIZE_SVGA 800x600 pixeles [iluminación media - día]
- **Dataset_Custom_3:** 213 Fotos tomadas a resolución FRAMSIZE_SVGA 800x600 pixeles [Variabilidad de escenarios reales].

Figura 5. Muestras de sets de imágenes Dataset Egocéntrico Real



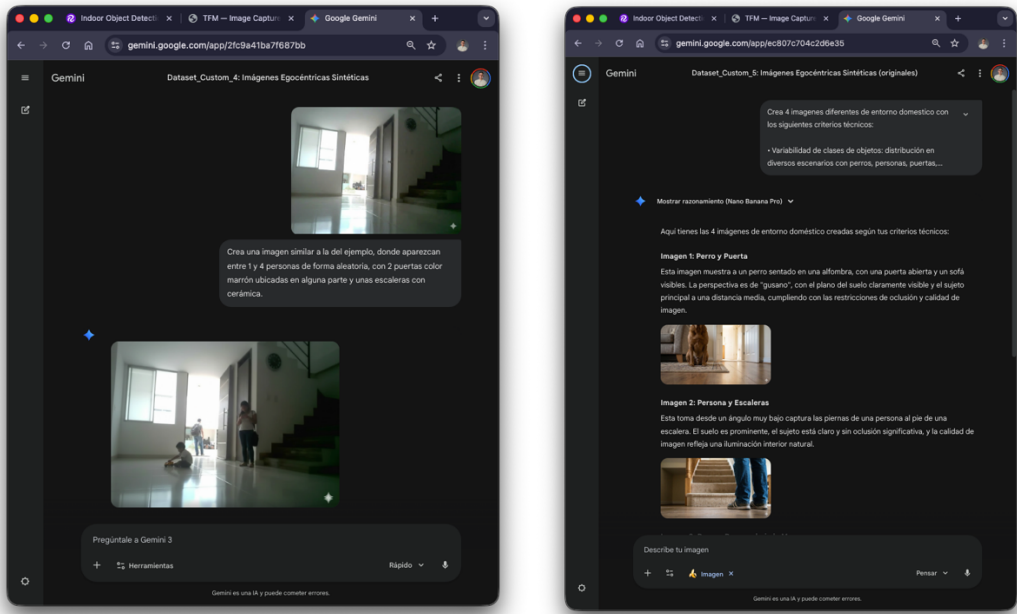
Nota: Cada set de imágenes fue cargado al proyecto propio: “Indoor Object Detection Custom” creado en la Plataforma Roboflow para su posterior etiquetado de anotaciones manual.

4.4.2. Dataset Egocéntrico Sintético

Estas imágenes fueron generadas sintéticamente a partir de algunas de las imágenes de los tres sets egocéntricos reales mediante procesamiento avanzado de Inteligencia Artificial basado en el modelo *NANO BANANA PRO* de Google (2025). El procedimiento consistió en el cargue de imágenes reales de los escenarios reales con presencia de las cinco clases de objetos y la aplicación de una estrategia de edición por *prompting* para generar variaciones en iluminación, cantidad de objetos en la escena, variabilidad en la ubicación de los objetos, ángulos y perspectivas, formas, colores y tonalidades de los objetos, entre otras. En la figura 6 se muestra un ejemplo del procedimiento. La figura 7 contiene algunas muestras de imágenes recopiladas desde esta fuente, las cuales fueron obtenidas en dos sets o subconjuntos de imágenes:

- **Dataset_Custom_4:** 32 imágenes sintéticas a resoluciones variables [escenarios creados a partir de modificaciones leves al escenario del entorno real].
- **Dataset_Custom_5:** 102 imágenes sintéticas a resoluciones variables [escenarios creados totalmente desde cero con instrucciones basadas en los criterios técnicos objetivo].

Figura 6. Ejemplo de prompting para generación de imágenes sintéticas



Nota: Captura de interfaz GEMINI de Google (2025) con uso del modelo NANO BANANA PRO

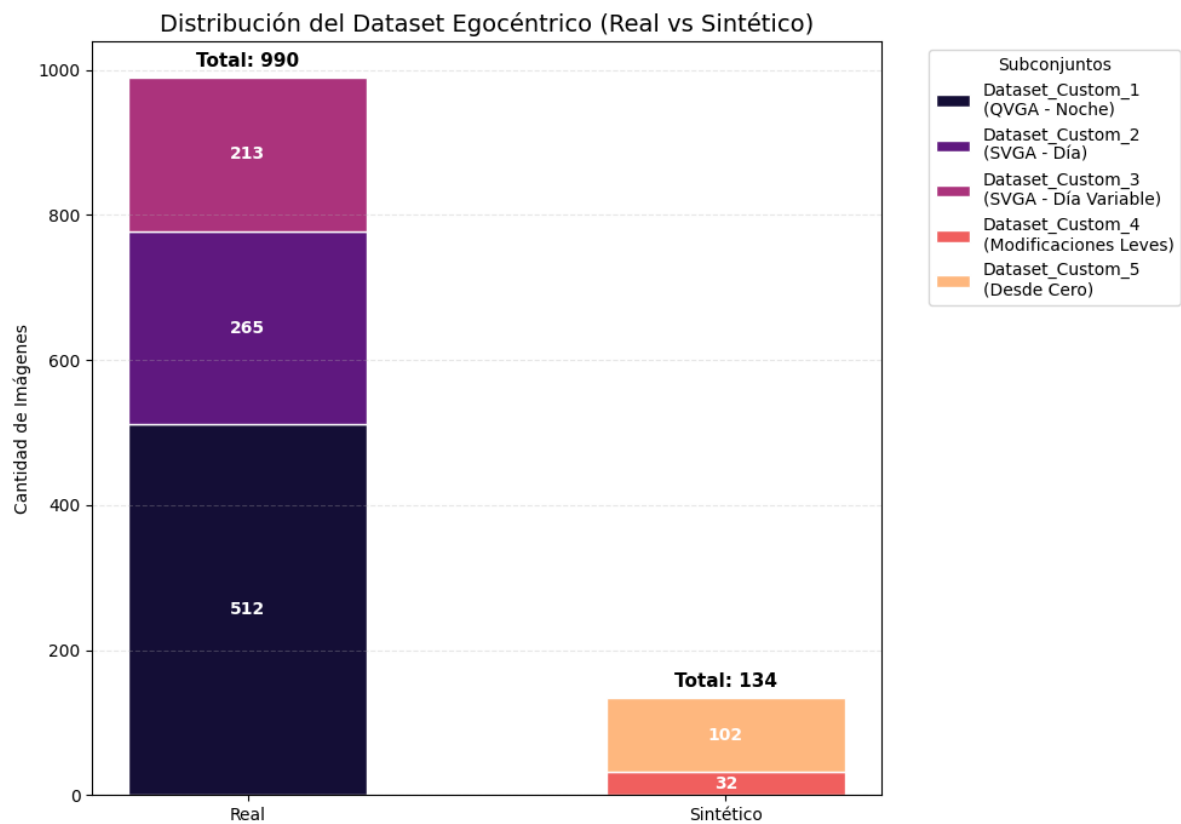
Figura 7. Muestras de sets de imágenes Dataset Egocéntrico Sintético



Nota: Cada set de imágenes fue cargado al proyecto propio: “Indoor Object Detection Custom” creado en la Plataforma Roboflow para su posterior etiquetado de anotaciones manual.

La distribución final del dataset egocéntrico, conformado por imágenes reales y sintéticas, se presenta en la Figura 8. El conjunto integra un total de 1.124 imágenes con una proporción de 7:1 entre capturas reales y sintéticas. Esta relación garantiza la integridad del escenario de operación real propuesto para el LCMR en esta investigación, evitando alteraciones en el contexto de entrenamiento, pero agregando variabilidad de escenas que promuevan una generalización eficaz, así como el mayor equilibrio natural de clases.

Figura 8. Distribución del Dataset Egocéntrico (Real vs Sintético)



4.5.Preparación del Dataset Compuesto: Anotaciones

Para la gestión integral de los datos, se empleó la plataforma *Roboflow*, un ecosistema especializado en el ciclo de vida de modelos de Inteligencia Artificial (IA) que facilita la compilación, anotación y despliegue de conjuntos de datos, así como el entrenamiento y despliegue de modelos de *Deep Learning*. Específicamente, se utilizó la funcionalidad *Roboflow Universe* para centralizar un dataset compuesto por los cinco subconjuntos previamente descritos, alcanzando un total de 1.124 imágenes.

El proceso se ejecutó siguiendo las etapas que se detallan a continuación:

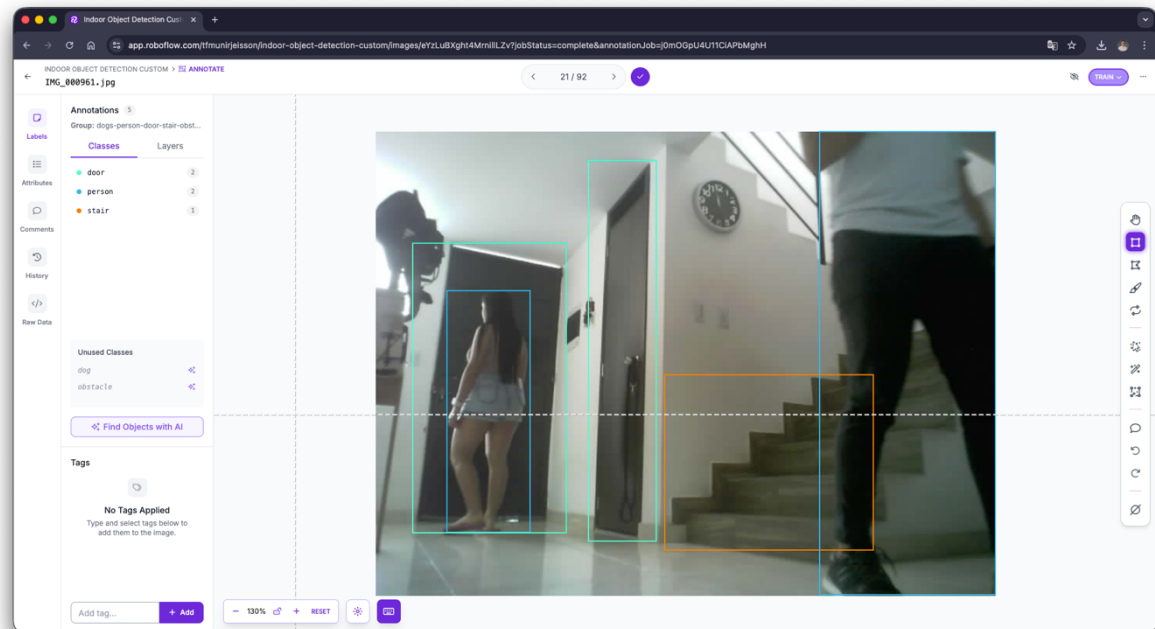
- **Carga y Consolidación:** Se integraron las muestras de los cinco subconjuntos en un proyecto único para garantizar la variabilidad de clases en diversos escenarios.
- **Anotación Manual:** Mediante la herramienta de etiquetado de la plataforma, se procedió a la asignación manual de cajas contenedoras (bounding boxes) objeto por objeto. Este proceso se rigió estrictamente por los criterios técnicos de ajuste estricto (tight) y anotación modal, asegurando que el borde inferior de cada caja coincidiera con el límite visual del objeto (ver Figura 9).
- **Distribución de Clases:** El proceso de etiquetado permitió cuantificar la presencia de cada clase dentro del dataset, cuya distribución se detalla en la Tabla 9 para facilitar su análisis comparativo.
- **Versionado y descarga:** Se generó una versión denominada "Original". Esta versión se caracteriza por carecer de divisiones (*splits*), preprocesamientos o técnicas de aumentación de datos (*data augmentation*). El objetivo de mantener el dataset en su estado primigenio es permitir un Análisis Exploratorio de Datos (EDA, por sus siglas en inglés) fidedigno, el cual se describirá en secciones posteriores. Para garantizar la versatilidad en el entrenamiento de futuros modelos, los datos se exportaron en los formatos *YOLO*¹, *COCO*² y *TFRecord*³.

¹ YOLO (*You Only Look Once*): Arquitectura de red neuronal convolucional diseñada para la detección de objetos en tiempo real.

² COCO (*Common Objects in Context*): Formato estándar y conjunto de datos a gran escala diseñado para tareas de detección, segmentación y etiquetado de objetos.

³ TFRecord (*TensorFlow Record*): Formato de almacenamiento binario optimizado para el ecosistema de TensorFlow.

Figura 9. Anotación de labels de clases en imágenes



Nota: Captura de interfaz Roboflow Annotations para etiquetado de labels de clases en imágenes

4.6. Análisis Exploratorio de Datos

El éxito de los modelos de detección de objetos en entornos de interiores depende, en gran medida, de la comprensión de la naturaleza del conjunto de datos. Por ello, se ha implementado un proceso de EDA, a través de un Jupyter Notebook en la estación de trabajo local, estructurado en 6 pasos diseñados de la siguiente manera:

- Paso 1: Descomprimir el Dataset y Mapear Imágenes
- Paso 2: Análisis Técnico del Dataset
- Paso 3: Análisis de Distribución de Clases
- Paso 4: Análisis Geométrico de Bounding Boxes
- Paso 5: Análisis de la Complejidad de la Escena
- Paso 6: Inspección Visual Cualitativa

A continuación, se detallan cada uno de estos pasos y sus resultados.

4.6.1. Descomprimir el Dataset y Mapear Imágenes

En esta fase, se procesa el conjunto de datos consolidado, *Indoor Object Detection Custom*, el cual se obtiene en formato YOLO dentro de un archivo comprimido (.zip⁴). Tras la descompresión, se procede con la lectura del archivo *data.yaml* para identificar las clases registradas. Posteriormente, se efectúa una validación de los nombres de dichas clases mediante un proceso de normalización que incluye la conversión a minúsculas (lowercase), la transformación a singular y la gestión de excepciones, con el objetivo de prevenir inconsistencias técnicas.

Asimismo, se construye un mapeo global unificado de clases para garantizar la coherencia del modelo. Las imágenes se trasladan a un directorio definitivo denominado *</data>*, estructurado en las subcarpetas *</images>* y *</labels>*. En estas, se organizan las capturas y sus respectivas anotaciones siguiendo el mapeo global y una serialización de nombres que asegura el orden requerido por el proyecto. Finalmente, se genera el archivo *data.yaml* unificado junto con un registro de trazabilidad (*log*). El mapeo se detalla en la Tabla 9.

Tabla 9. Distribución de etiquetas por clase en el dataset consolidado

Clase de Objeto	Etiqueta (Label)	ID de Mapeo	Cantidad de Anotaciones
Perros (mascotas)	`dog`	0	485
Puertas	`door`	1	541
Obstáculos (muebles, sillas, mesas, etc.)	`obstacle`	2	882
Personas	`person`	3	592
Escaleras	`stair`	4	509
Total			3,009

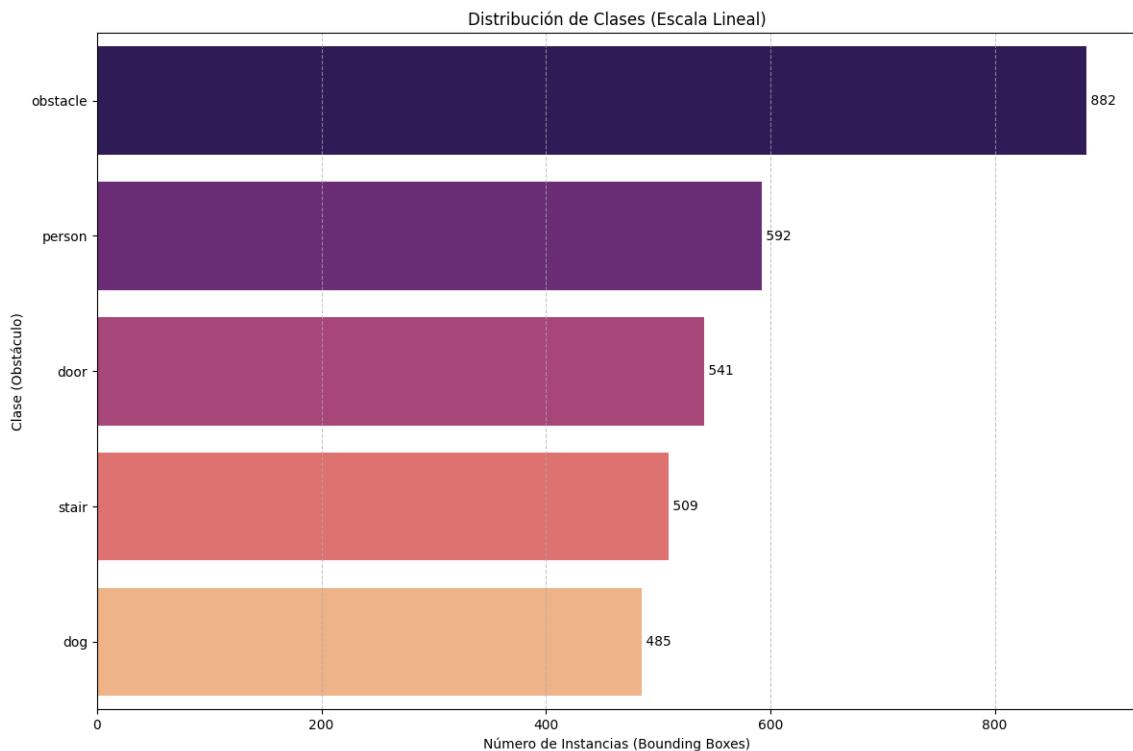
Nota: La cantidad de anotaciones es superior al número de imágenes debido a la presencia de múltiples objetos en una misma captura.

La figura 10 muestra una distribución moderadamente equilibrada: *obstacle* (882, 29.3%) lidera; *dog* (485, 16.1%) es la clase minoritaria. Se tiene un Ratio max/min = 1.82:1, considerado leve y no se espera sesgo severo de los modelos. Aun así, *obstacle* casi duplica a *dog*, y para navegación robótica, *dog* (agente biológico impredecible) requiere *recall* alto. A partir de estos datos se concluye que para el preprocesamiento se debe aplicar *augmentation*

⁴ ZIP (Zone Information Protocol) carpeta contenedora comprimida.

uniforme y considerar leve *oversampling* de *dog*. Lo anterior también implica que en el entrenamiento se debe monitorear *mAP per-class* y aplicar *Class weights* inversamente proporcionales a frecuencia dado el bajo desbalance.

Figura 10. Distribución de Clases en el Dataset



4.6.2. Análisis Técnico del Dataset

Antes de proceder con el análisis del contenido semántico del conjunto de datos, es prioritario verificar la calidad técnica de la información a nivel de archivos e imágenes dentro del *dataset*. En este paso, se analizan las resoluciones para garantizar la consistencia dimensional y se ejecutan controles de integridad (*sanity checks*) orientados a la detección de archivos corruptos, la validación de formatos de color (*RGB*⁵) y la identificación de duplicados exactos que puedan comprometer la integridad de los datos. Así mismo, se calculan las estadísticas de píxeles, específicamente la media y la desviación estándar por canal RGB, parámetros que

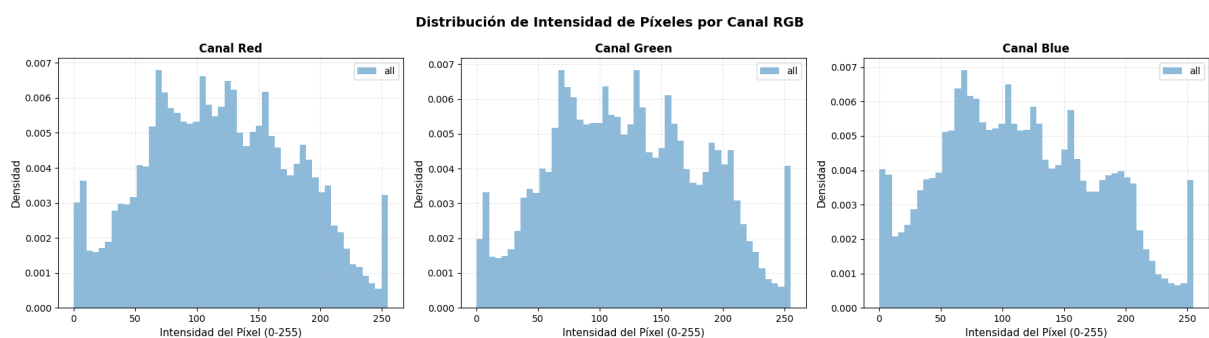
⁵ RGB, Red – Green – Blue

resultan importantes para la normalización en entornos de desarrollo como *PyTorch* o *TensorFlow*.

En cuanto a la integridad técnica y la redundancia de datos, se obtuvieron resultados satisfactorios que confirman la calidad del conjunto de datos. El dataset se encuentra exento de duplicados y la totalidad de las muestras han sido validadas como legibles y correctamente configuradas en el formato de color RGB. De este modo, se elimina el riesgo de filtración de datos (*data leakage*), lo cual permite que la segmentación en los subconjuntos de entrenamiento, validación y prueba se realice de manera directa, sin requerir una deduplicación previa. Finalmente, la validez técnica de las imágenes asegura que no sea necesario implementar procedimientos adicionales para la gestión de errores durante las fases posteriores del proyecto.

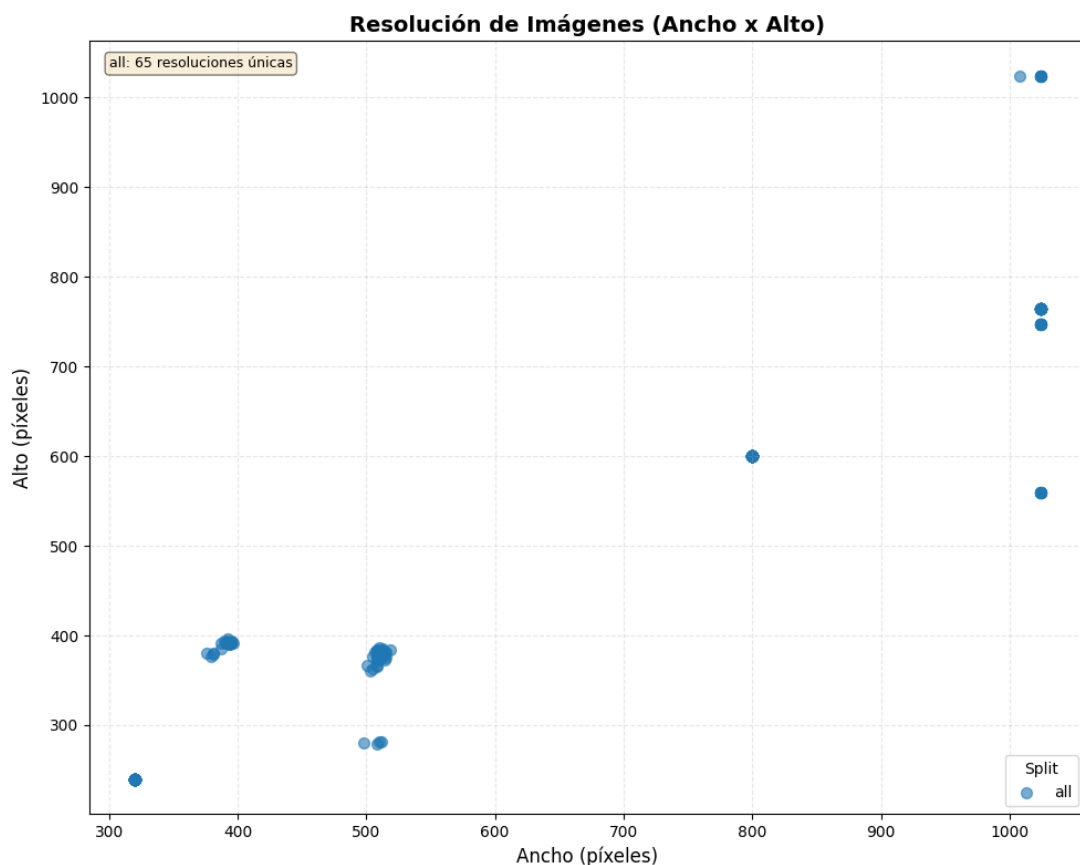
En la Figura 11 se detalla la distribución de intensidad de píxeles, con picos principales en el rango de 50 a 150 en los 3 canales señala que las imágenes contienen escenas de iluminación interior relativamente homogéneas. Comportamiento similar entre canales R, G y B. Sin picos pronunciados en valores máximo lo que sugiere que no hay sobreexposición. Esta baja varianza podría limitar la generalización a condiciones extremas de luz, baja y alta. Se requiere *augmentation* de brillo, contraste y gamma para ampliar cobertura del espacio de iluminación y mejorar la robustez. Las estadísticas de canales RGB, estandarizado, generan un $RGB_{mean} = [0.4557, 0.4777, 0.442]$ ligeramente inferior a 0.5 y un $RGB_{std} = [0.2281, 0.2341, 0.2410]$ uniforme entre canales. Estos datos serían útiles al entrenar un modelo nuevo desde cero (*from scratch*), pero al usar como base modelos pre-entrenados, los valores de *mean* y *std* para canales RGB están cerca a los estándar de *ImageNet* y *COCO*; por tanto, no es necesaria la normalización para *Transfer Learning*.

Figura 11. Distribución de Intensidad de Píxeles por canal RGB



La extracción de dimensiones, detallada en la Figura 12, reveló que el conjunto de datos cuenta con 65 resoluciones únicas, las cuales oscilan entre 320×240 y 1024×1024 , con una mayor frecuencia en los rangos de 500×370 y 800×600 . Si bien esta diversidad favorece la robustez del modelo, se requiere un proceso de redimensionamiento para garantizar la conformación de lotes (*batches*) uniformes. En este sentido, se establece una resolución máxima de 640×640 , óptima para arquitecturas de la familia YOLO, y una mínima de 192×192 para la ejecución de entrenamientos multiescala orientados a dispositivos de bajo consumo (*TinyML*). Este enfoque permite implementar técnicas de reajuste progresivo (*progressive resizing*), lo cual incrementa la resiliencia del sistema ante entradas de baja resolución y optimiza el rendimiento general de los modelos.

Figura 12. Extracción de resoluciones de Imágenes (Ancho x Alto)



4.6.3. Análisis de Distribución de Clases

Se analiza la densidad de imágenes, como se observa en la Figura 13, tiene una distribución homogénea en la que todas las clases aparecen en más del 34% de las capturas (rango de 391 a 599 imágenes). Esta ubicuidad favorece un aprendizaje equilibrado de los modelos, ya que

no se identificaron clases raras o confinadas a un número reducido de muestras. Asimismo, el estudio de la matriz de co-ocurrencia, mostrado en la Figura 14, revela que el conjunto de datos refleja escenas naturales multi-clases de alto valor contextual. Destacan asociaciones frecuentes entre las clases *obstacle-dog* (231 imágenes), *person-stair* (207 imágenes) y *door-stair* (202 imágenes), lo cual enriquece el aprendizaje de contextos realistas.

Figura 13. Densidad de Imágenes vs. Objetos por Clase

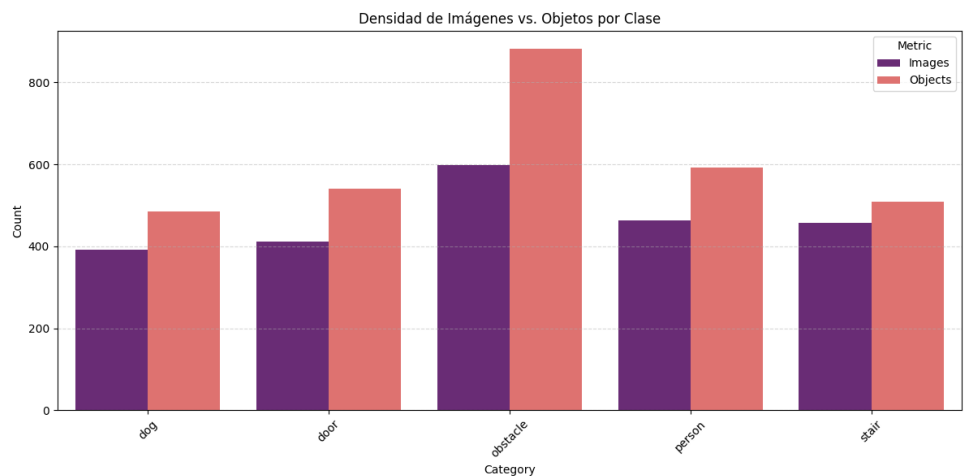
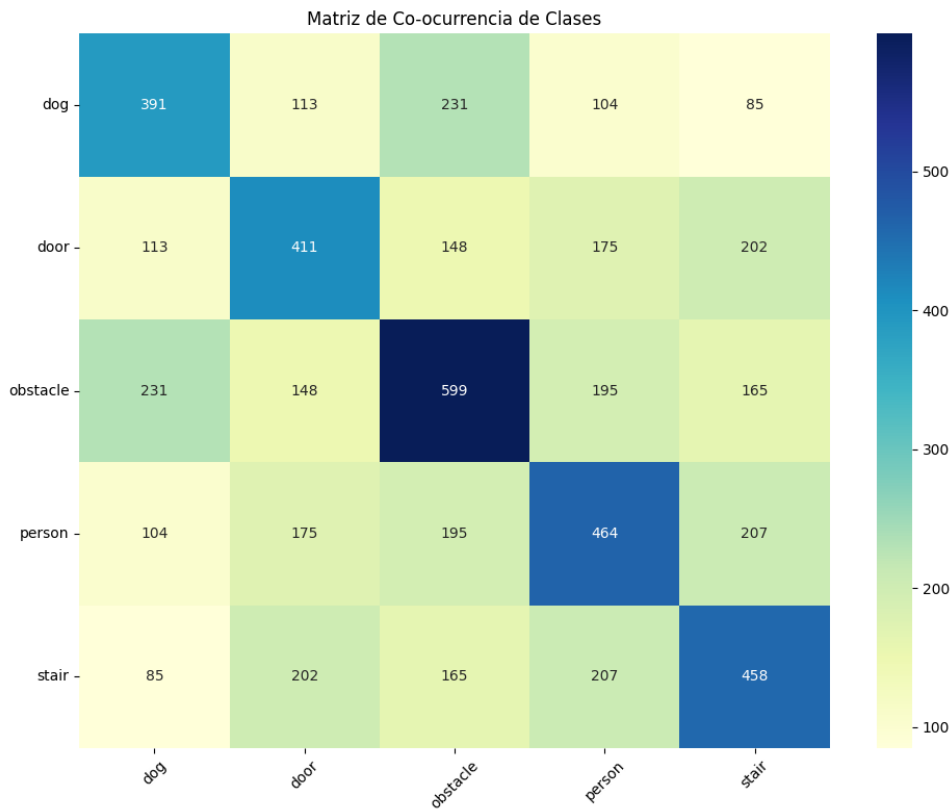
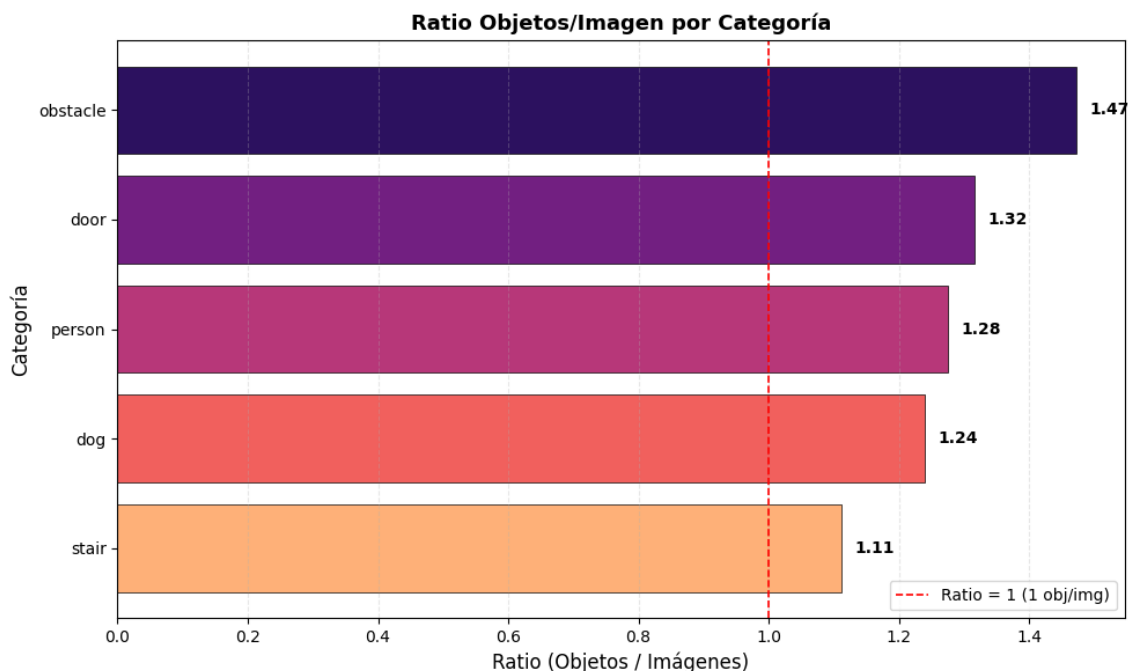


Figura 14. Matriz de Co-ocurrencia de clases



Finalmente, en la figura 15 se visualiza el cálculo del *ratio* de objetos por clase arrojó valores uniformemente reducidos, oscilando entre 1,11 y 1,47 instancias por imagen. Esta baja densidad de objetos por clase sugiere un escenario simplificado para el proceso de supresión no máxima (NMS, por sus siglas en inglés: *Non-Maximum Suppression*), al existir un solapamiento mínimo entre instancias de la misma categoría. Esta característica resulta particularmente favorable para el despliegue de los modelos en hardware con recursos limitados, como el ESP32-S3, permitiendo configurar un umbral de detecciones máximas conservador (entre 10 y 15) sin comprometer la precisión del sistema.

Figura 15. *Ratio de Objetos/Imagen por clase*



4.6.4. Análisis Geométrico de Bounding Boxes

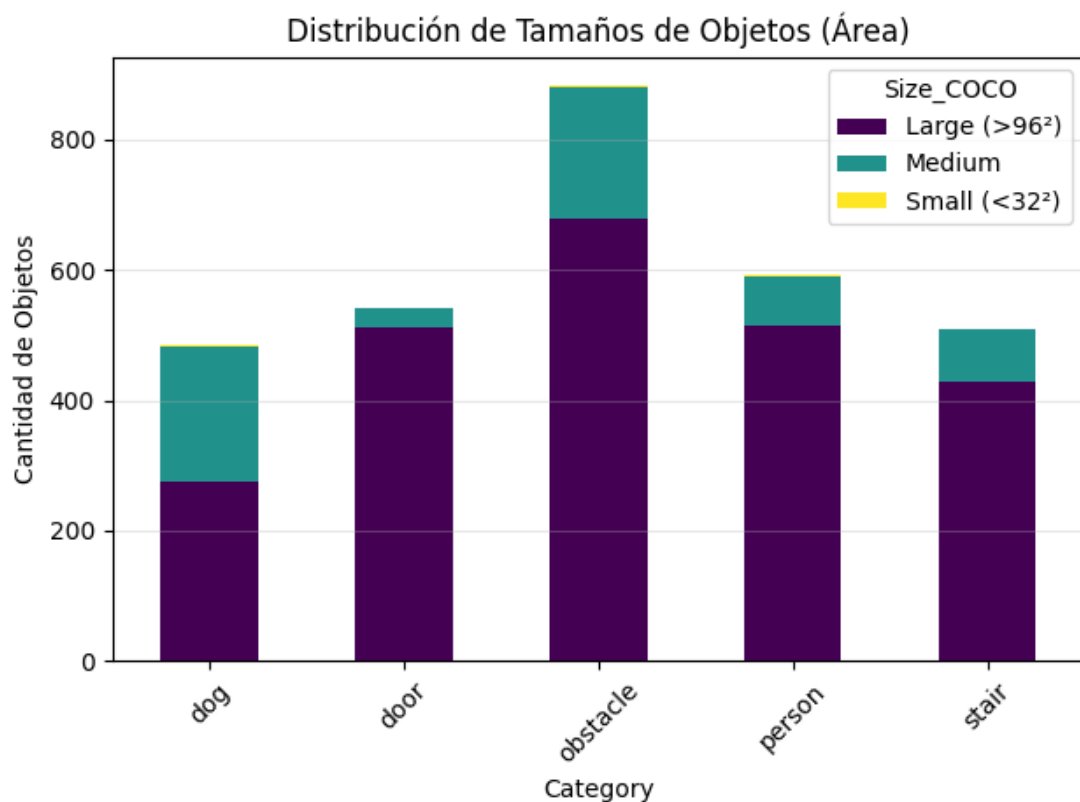
En este paso se realizó un análisis de la geometría de las 3,009 anotaciones que componen el dataset. Los resultados arrojaron una gran diversidad de formas y dimensiones como lo comprueban la Figura 16 y la Tabla 10, destacando que las clases *door* y *stair* presentan las mayores áreas medias (60,073 px² y 59,968 px², respectivamente). Por su parte, la clase *dog* exhibió la mayor variabilidad, con una distribución sesgada donde la relación entre la media y la mediana es de aproximadamente 3.5:1, lo que sugiere la presencia de valores atípicos (*outliers*) de gran tamaño.

Tabla 10. Geometría de Bounding Boxes extraída de anotaciones

Clase	Cantidad	Área				% de Tamaños COCO (pixeles)		
		Promedio	Mediana	Ratio	Varianza	Grande ($> 96^2$)	Mediano	Pequeño ($< 32^2$)
dog	485	38819.12	11056.00	1.04	0.50	56.49	42.58	0.82
door	541	60072.54	37585.25	0.44	0.22	94.64	5.36	0
obstacle	882	39128.28	19167.99	0.75	0.38	76.87	23.02	0.11
person	592	47895.17	32768.83	0.50	0.26	86.82	12.84	0.34
stair	509	59968.08	39005.87	1.21	0.52	15.91	15.91	0

En cuanto a la distribución de tamaños bajo el estándar COCO, se observó una predominancia de objetos grandes (*Large* $> 96^2$), representando el 94.6 % en la clase *door* y más del 76 % en *obstacle*, *person* y *stair*. Esta composición es coherente con capturas en interiores a corta distancia. Por otro lado, la clase *dog* resultó ser la más heterogénea, con casi la mitad de sus instancias clasificadas como de tamaño mediano (*Medium*). Cabe destacar que los objetos pequeños (*Small*) son marginales, representando menos del 1 % en todas las categorías.

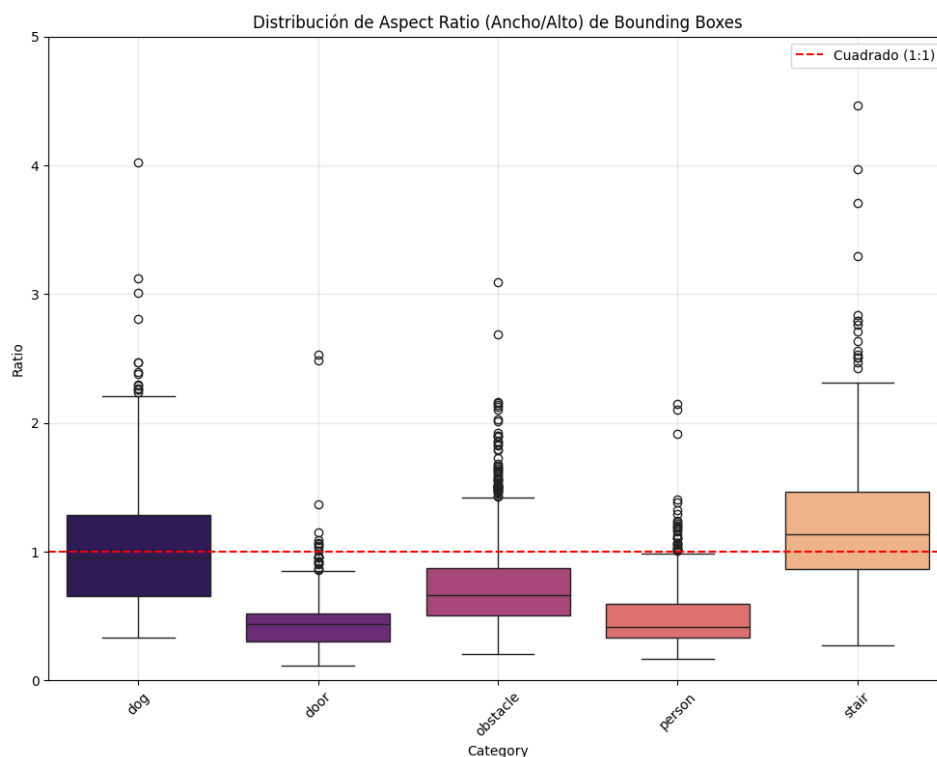
Figura 16. Distribución de Tamaños de Objetos (Área)



Respecto al factor de forma (*aspect ratio*), representado en la Figura 17, se identificaron tendencias geométricas claras:

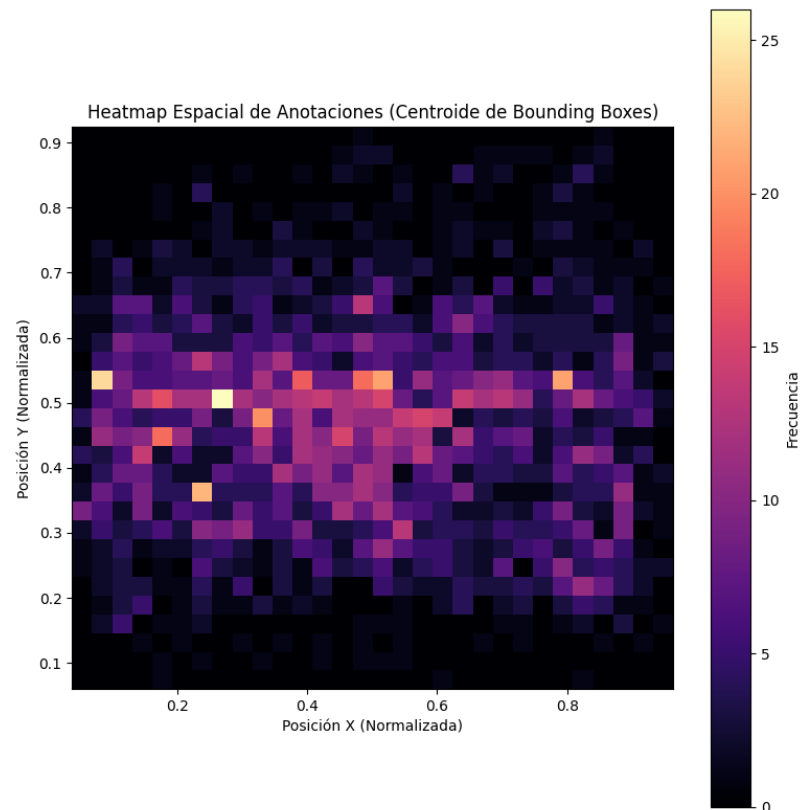
- **Verticales:** Las clases *door* (0.44) y *person* (0.50) muestran una orientación marcadamente vertical.
- **Horizontales:** La categoría *stair* tiende a la horizontalidad (1.21), con una alta dispersión.
- **Cuadrangulares:** La clase *dog* (1.04) presenta una forma cercana al cuadrado.

Figura 17. Distribución de Aspect Ratio (Ancho/Alto) de Bounding Boxes



El análisis del mapa de calor espacial, mostrado en la Figura 18, reveló una concentración de centroides en la franja horizontal central (eje Y entre 0.3 y 0.6). Esta distribución es consistente con una cámara situada a la altura de un robot, donde los objetos de interés se alinean con el horizonte. Para normalizar estos datos independientemente de las 65 resoluciones distintas del dataset, se extrajeron las dimensiones en el rango [0,1], facilitando así la comparabilidad de la información.

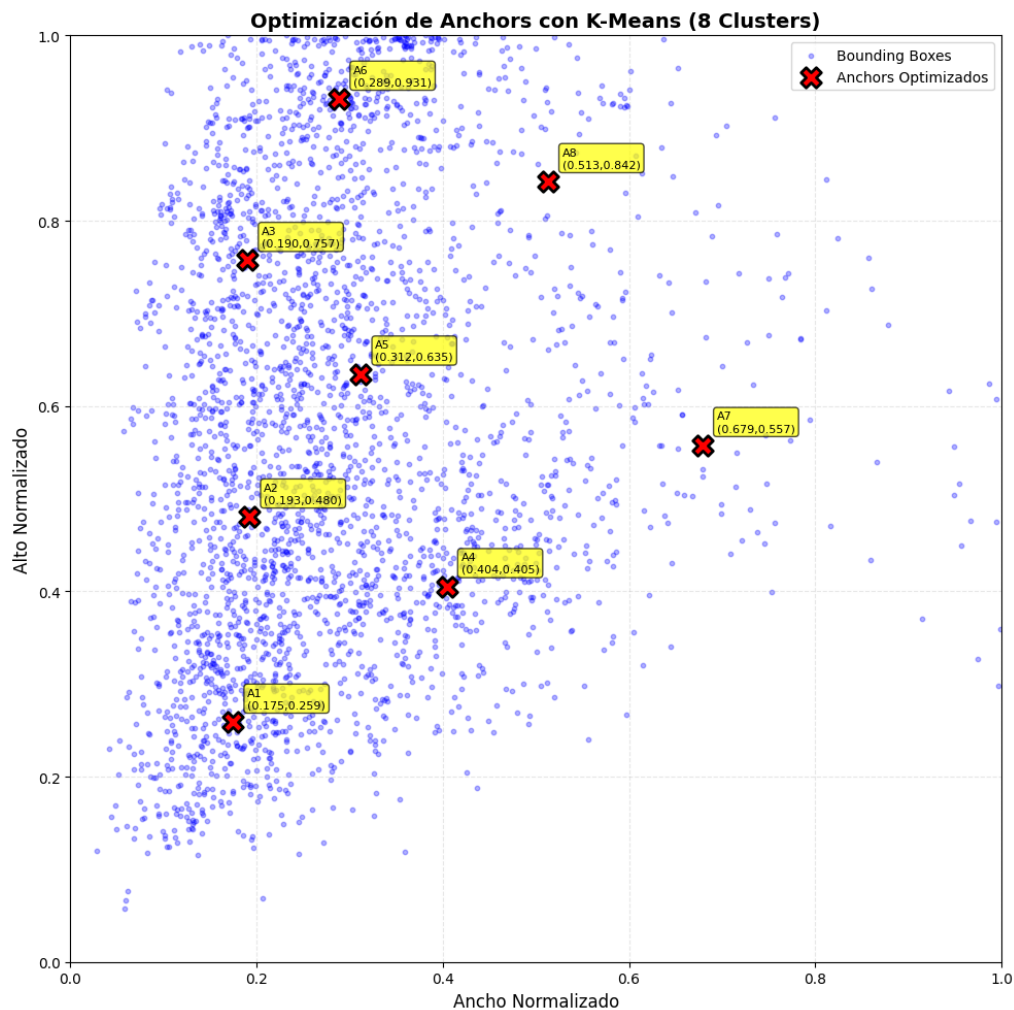
Figura 18. *Heatmap Espacial de Anotaciones por centroide de Bounding Boxes*



Finalmente, se empleó el algoritmo K-Means para calcular 8 clústeres de anclajes (*anchors*) optimizados. Estos varían desde dimensiones reducidas (0.175×0.259) hasta configuraciones de gran escala (0.513×0.842). La configuración resultante se exportó en un archivo *YAML*⁶, estructurada en tres escalas (P3, P4 y P5), lista para ser integrada en modelos que sean basados en anclajes de las cajas delimitadoras (*anchors-based*) como algunos relativamente antiguos de la familia YOLO. Sin embargo, para el despliegue en TinyML, es necesario reescalar estos parámetros proporcionalmente a la resolución de inferencia final.

⁶ *YAML* (*YAML Ain't Markup Language*): Formato estándar de representación de datos estructurados, optimizado para la legibilidad humana y la configuración de sistemas de software.

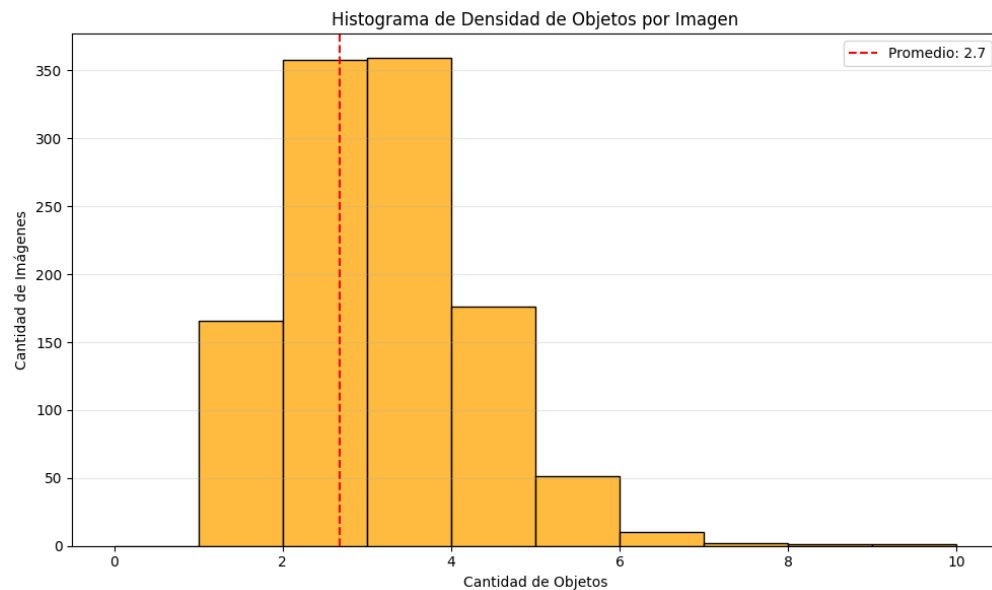
Figura 19. Identificación de clústeres de Anchors con K-Means



4.6.5. Análisis de la Complejidad de la Escena

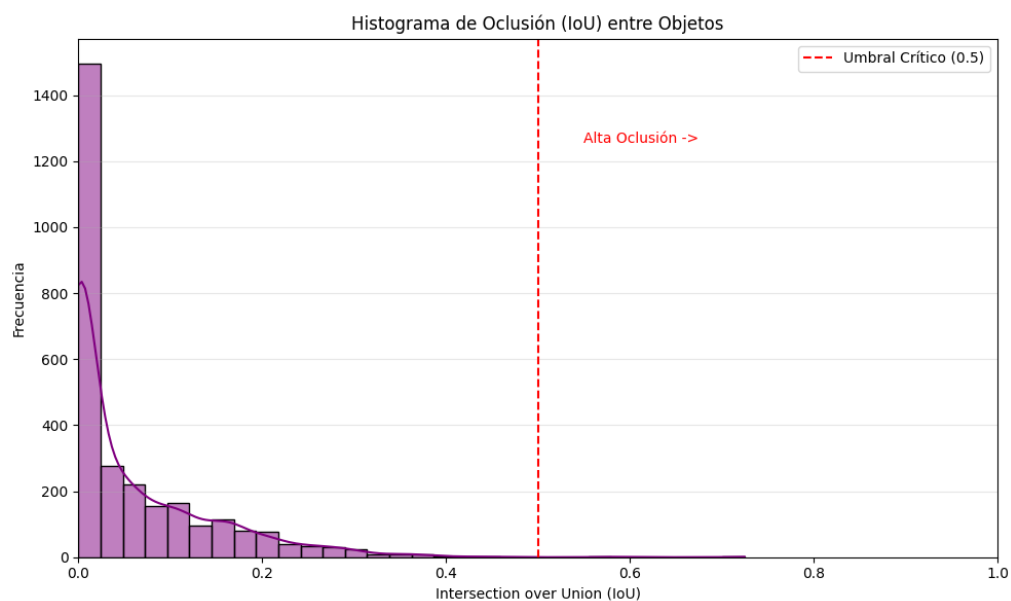
Este paso evaluó la densidad de objetos por imagen mediante el uso de histogramas de distribución. El análisis, detallado en Figura 20, reveló una distribución compacta y centrada, con un promedio de 2.7 objetos por captura y un máximo acotado de 10 unidades, sin presencia de valores atípicos extremos. Esta carga de detección, moderada y uniforme, resulta altamente favorable para la implementación en el ESP32-S3, ya que asegura un consumo de recursos predecible. Por consiguiente, se ha determinado que establecer un límite de entre 10 y 15 detecciones máximas es suficiente para cubrir los requerimientos de los modelos sin comprometer la estabilidad de la memoria.

Figura 20. *Densidad de Objetos por Imagen*



De manera simultánea, se examinó el nivel de oclusión entre objetos a través del cálculo del índice de Intersección sobre la Unión (IoU). Los resultados, en la Figura 21, muestran una distribución exponencialmente decreciente donde la gran mayoría de los pares de objetos presentan un IoU inferior a 0.1, y los casos con un solapamiento superior al umbral crítico de 0.5 son prácticamente inexistentes. Esta baja oclusión generalizada confirma la integridad de las anotaciones y permite el uso de algoritmos de supresión no máxima (NMS) simplificados, lo cual es técnica y operacionalmente viable para entornos de TinyML.

Figura 21. *Oclusión (IoU) entre Objetos*

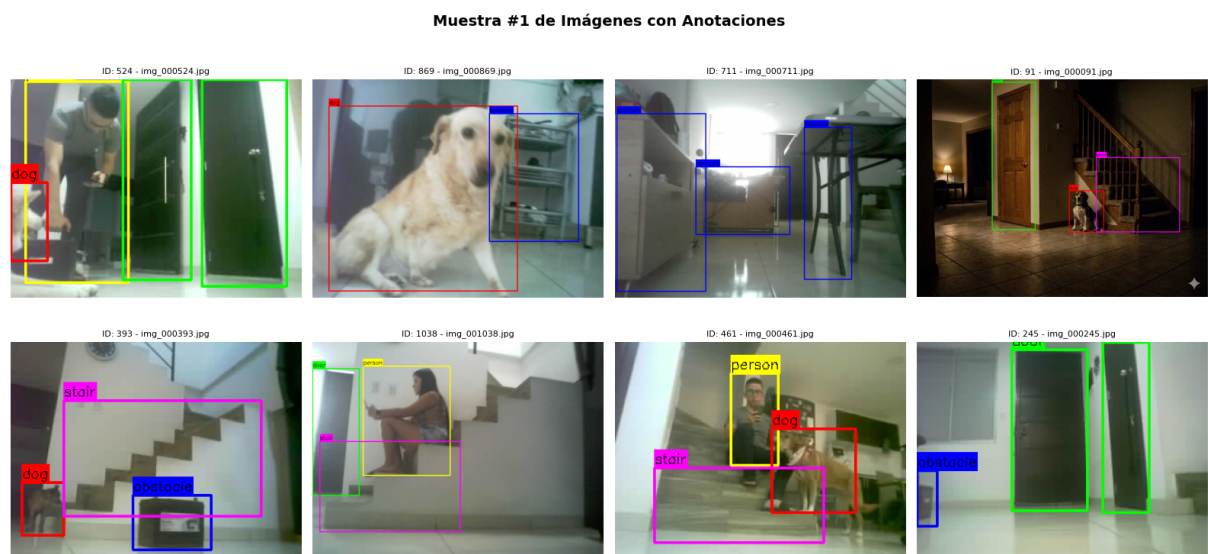


Finalmente, el resumen numérico basado en la inspección de 1,124 imágenes confirmó que la totalidad del conjunto de datos contiene al menos una instancia de interés, careciendo de imágenes de fondo o vacías (background). Si bien el promedio bajo de objetos por imagen beneficia el rendimiento en hardware limitado, es probable que la ausencia de muestras negativas pueda inducir al modelo a generar detecciones erróneas en escenas sin objetos. Por lo tanto, en casos como este es necesaria la incorporación de muestras de fondo artificiales durante el preprocesamiento para mitigar potenciales falsos positivos y robustecer la capacidad de discriminación del sistema.

4.6.6. Inspección Visual Cualitativa

Finalmente, se ejecutó un control de integridad mediante la inspección visual de muestras aleatorias del conjunto de datos. Los resultados, plasmados en las Figuras 22, 23 y 24, confirmaron que las anotaciones, después del mapeo inicial, son coherentes y presentan cuadros delimitadores (*bounding boxes*) ajustados con precisión a la morfología de los objetos. También se valida, formalmente, que la muestra analizada refleja una amplia diversidad de escenarios interiores naturales —tales como habitaciones, pasillos, escaleras y accesos— Adicionalmente, se ratificó la heterogeneidad dimensional de las capturas, validando la consistencia de los datos ante diversas resoluciones de entrada.

Figura 22. Muestra 1 de Imágenes con Anotaciones

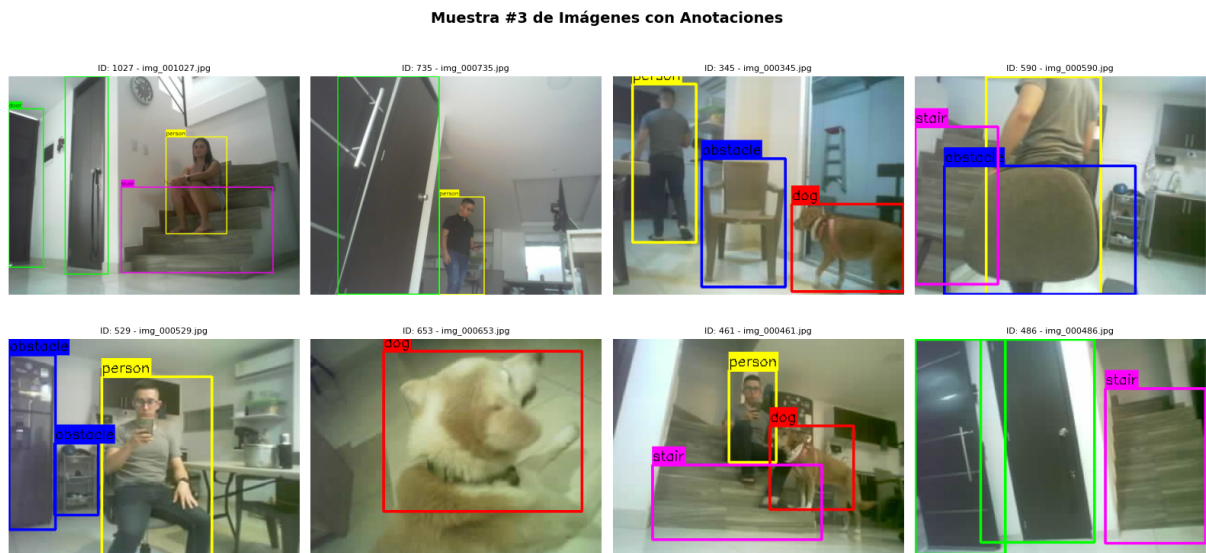


Por otro lado, el análisis orientado a la detección de anomalías técnicas arrojó resultados satisfactorios. No se identificaron cuadros delimitadores degenerados, tales como anotaciones minúsculas (inferiores a 15×15 píxeles) o de tamaño excesivo que cubrieran más del 95 % del marco de la imagen. Esta ausencia de artefactos de borde y de etiquetas inconsistentes certifica la excelente calidad del etiquetado.

Figura 23. Muestra 2 de Imágenes con Anotaciones



Figura 24. Muestra 3 de Imágenes con Anotaciones



4.7. Pipeline de Preprocesamiento

Este apartado describe el protocolo diseñado para transformar el conjunto de datos original en una base optimizada para arquitecturas de detección de objetos en sistemas embebidos. Tras el Análisis Exploratorio de Datos (EDA) que evidenció una alta calidad en las etiquetas, pero una varianza lumínica limitada y la ausencia de muestras de fondo, se estructuró un flujo de procesamiento en cinco fases fundamentales.

4.7.1. Mitigación de Falsos Positivos mediante Inyección de Fondo

El EDA determinó que la totalidad de las imágenes originales contenían al menos un objeto anotado, factor que incrementa el riesgo de "alucinación" de objetos en espacios vacíos dentro de un entorno doméstico. En consecuencia, la literatura reciente demuestra que integrar entre el 10% y el 15% de muestras de fondo (De 112 a 168 imágenes, en el caso actual), permiten al modelo reconocer el ruido visual de interiores como una clase de relleno, optimizando la precisión al reducir la tasa de falsos positivos (Lee et al., 2025; Glez-Peña et al., 2023;). Más aún en dispositivos de borde (*Edge AI*) como el ESP32-S3, donde se manejan resoluciones bajas y los píxeles de fondo pueden biselarse y parecerse a los objetos de interés.

- **Composición y Criterios Técnicos de las Muestras de Fondo:** Para asegurar la eficacia de esta inyección, las imágenes son capturadas en el entorno desde la perspectiva egocéntrica del robot, centrándose en texturas y geometrías permanentes del contexto real. Las tomas incluyen planos de tierra despejados (Ej. baldosas), intersecciones entre paredes y suelos para evitar confusiones con bordes de muebles, y espacios de transición vacíos. Se mantienen los mismos criterios técnicos generales del dataset original.
- **Metodología de Obtención y Validación de Integridad:** La generación de este subconjunto se realizó mediante dos vías complementarias: la captura real, utilizando la interfaz de control del robot para obtener ráfagas en zonas vacías, y la generación sintética mediante modelos de inteligencia artificial ajustados a una resolución de 640×640.

Finalmente, se garantizó que estas imágenes se estuviesen estrictamente "limpias" de etiquetas. La presencia de elementos no catalogados —como la sección de un mueble no

etiquetada— enviaría señales contradictorias al sistema, indicando erróneamente que un objeto de interés debe ser interpretado como fondo. Como resultado final se obtuvo:

- **Dataset_Background_1:** 87 Fotos tomadas desde la ESP32-S3 + CAM OV5640 a resolución FRAMSIZE_QVGA 800x600 pixeles [iluminación variable].
- **Dataset_Background_2:** 38 imágenes sintéticas a resoluciones variables [escenarios creados totalmente desde cero con instrucciones basadas en los criterios técnicos objetivo].

Para un total de 125 imágenes de fondo que representan un 10% del dataset completo (1,249 imágenes). En la Figura 25 se observa una muestra de estos sets.

Figura 25. Muestras de sets de imágenes de fondo para mitigar falsos positivos



Nota: Cada set de imágenes fue cargado al proyecto propio: “Indoor Object Detection Custom” creado en la Plataforma Roboflow para su posterior etiquetado de anotaciones manual.

4.7.1. Normalización Dimensional y Preservación de la Relación de Aspecto

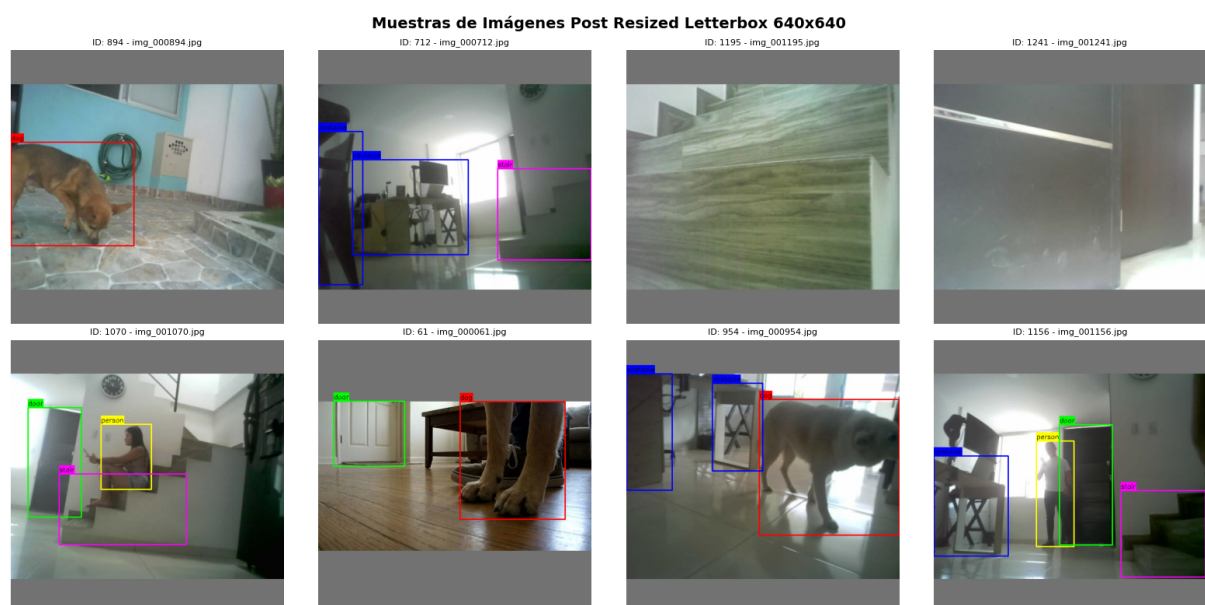
Ante la presencia de 65 resoluciones heterogéneas, se implementó un redimensionamiento uniforme a 640×640 píxeles. La fundamentación para esta decisión es la siguiente:

- **Compatibilidad con Transfer Learning:** La mayoría de cuerpos principales de arquitecturas (*backbones*) de modelos preentrenados (Ej. YOLOv11 o MobileNetV3) utilizan pesos optimizados para datasets como COCO o ImageNet con esta resolución; por lo tanto se puede aprovechar la extracción de características de bajo nivel ya aprendida por estos modelos.
- **Consistencia de Tensores y Lotes:** El EDA reveló 65 resoluciones heterogéneas, donde la media ronda los 640 píxeles. El reescalado uniforme es obligatorio para poder

construir tensores de entrada constantes y el procesamiento paralelo de lotes (*batches*) durante el entrenamiento.

- **Alineación con Arquitecturas:** Las redes neuronales con arquitecturas de Pirámide de Características (*Feature Pyramid Networks, FPN*) reducen la resolución en factores de $2^5 = 32$. El valor de 640 es múltiplo de 32, lo que garantiza que los mapas de características en las capas finales no tengan residuos de píxeles o distorsiones.
- **Preservación de detalles en la curación:** Al ser un dataset con muchos objetos grandes (*Large*) y medianos (*Medium*) según el estándar COCO, como el caso de las clases *door*, *person* y *obstacle*, el entrenamiento inicial a 640 asegura que los modelos aprendan la morfología detallada antes de ser sometido a restricciones de resolución mucho menores en el hardware embebido de la ESP32-S3.
- **Proyección hacia entrenamiento multiescala (*Multi-scale Training*):** Se implementa esta estrategia para entrenar con lotes a escalas del 50% y el 25% de la resolución base de 640 píxeles de forma aleatoria cada cierto número de lotes, para darle mayor robustez y capacidad a los modelos en el manejo de distintas resoluciones de inferencia e invarianza de escala. En el Capítulo cinco de Ingeniería de Modelos se profundizará más en este aspecto.

Figura 26. Muestra aleatoria de imágenes post resized letterbox



Se descartó el reescalado directo para prevenir la distorsión geométrica en clases de orientación vertical, como *person* y *door*. En su lugar, se aplicó la técnica de *Letterboxing*, la cual preserva la relación de aspecto original mediante la adición de un relleno (*padding*) constante de color gris ($RGB[114,114,114]$) para promover la neutralidad de la activación y el sesgo de borde por contraste alto, manteniendo la integridad morfológica necesaria para la extracción de características en capas convolucionales profundas. La Figura 26 presenta una muestra de algunas imágenes después de este proceso de reescalado.

4.7.2. División Estratificada del Conjunto de Datos

Para la construcción del dataset final, se realizó una división estratificada con una distribución del 70% para entrenamiento (*train*), 15% para validación (*valid*) y 15% para prueba (*test*). Esta técnica es esencial para mitigar el desbalance identificado en la clase *dog* (*ratio* de 1.82:1), garantizando que la distribución de probabilidad de las clases $P(C)$ permanezca invariante entre subconjuntos y evitando así sesgos en la evaluación del Recall en agentes biológicos críticos para la navegación de un LCMR.

Bajo este marco, se evaluaron dos enfoques de estratificación: la **Presencia por imagen** (multi-label), que busca mantener la proporción de combinaciones de etiquetas, y las **Instancias por clase**, que prioriza el equilibrio en el conteo total de cuadros delimitadores (bboxes) por categoría. Se seleccionó este último criterio como pilar de la ingeniería de datos para asegurar que las clases con menor frecuencia cuenten con una representación suficiente en los conjuntos de validación y prueba, lo que garantiza la fiabilidad estadística de las métricas de precisión.

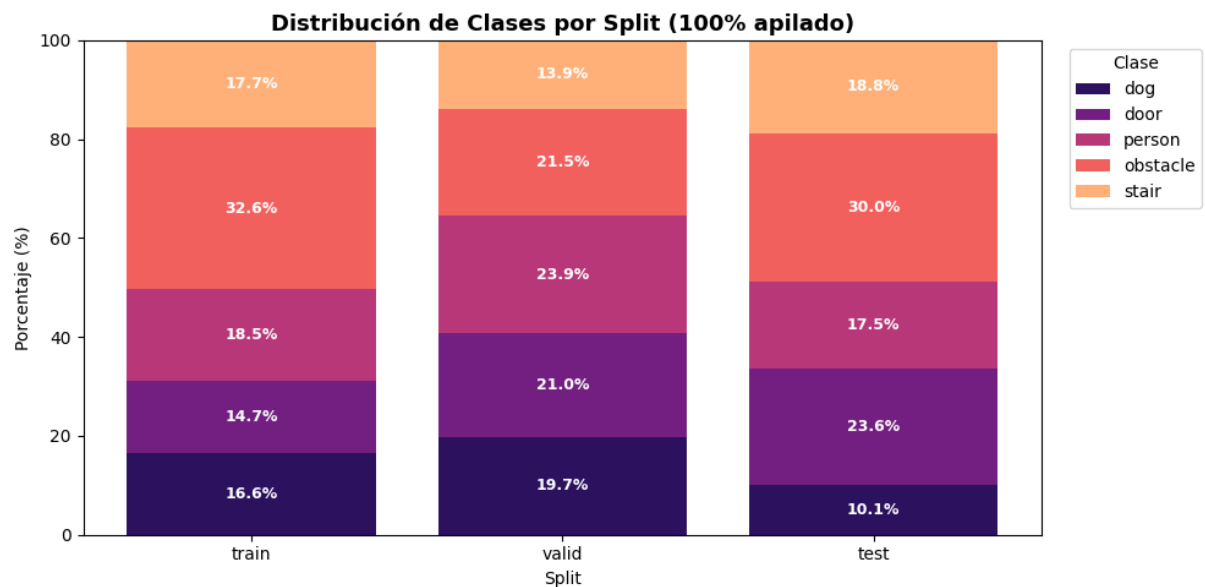
La ejecución de este protocolo resultó en una distribución balanceada de 1,249 imágenes totales, detalladas en la Tabla 11. El conjunto de entrenamiento se consolidó con 874 imágenes (1,671 anotaciones), mientras que los sets de validación y prueba disponen de 188 y 187 imágenes respectivamente. Así mismo, se mantuvo la inyección controlada de 125 muestras de fondo (background), distribuidas proporcionalmente (87 en entrenamiento, 19 en validación y 19 en prueba).

Tabla 11. *Distribución de divisiones en el dataset*

Split	Anotaciones						Imágenes		
	dog	door	person	obstacle	stair	total	Con objetos	Background	Total
Train	277	245	309	545	295	1,671	787	87	874
Valid	150	160	182	164	106	762	169	19	188
Test	58	136	101	173	108	576	168	19	187

La consistencia lograda mediante este proceso de estratificación se visualiza en la Figura 27, que detalla la Distribución de Clases por *Split* (100% apilado). En esta representación se evidencia cómo se preserva la proporción de cada clase en las tres particiones, manteniendo la clase obstacle como la de mayor frecuencia (entre el 21.5% y 32.6%) y asegurando que la clase crítica dog conserve una presencia estadística estable —oscilando entre el 10.1% y el 19.7%— para garantizar una evaluación de métricas de desempeño robusta y comparable en todos los subconjuntos.

Figura 27. *Muestra aleatoria de imágenes post resized letterbox*



4.7.3. Estrategia de Aumento de Datos (Augmentation)

La distribución de clases en el Train Set, en Figura 28, se mantuvo semejante a la mostrada en el Dataset completo. Caso similar sucede con el ratio de objetos/imagen por clase y la matriz de co-ocurrencia de clases, mostradas en las Figuras 29 y 30, respectivamente.

Figura 28. *Distribución de Clases de Train Set*

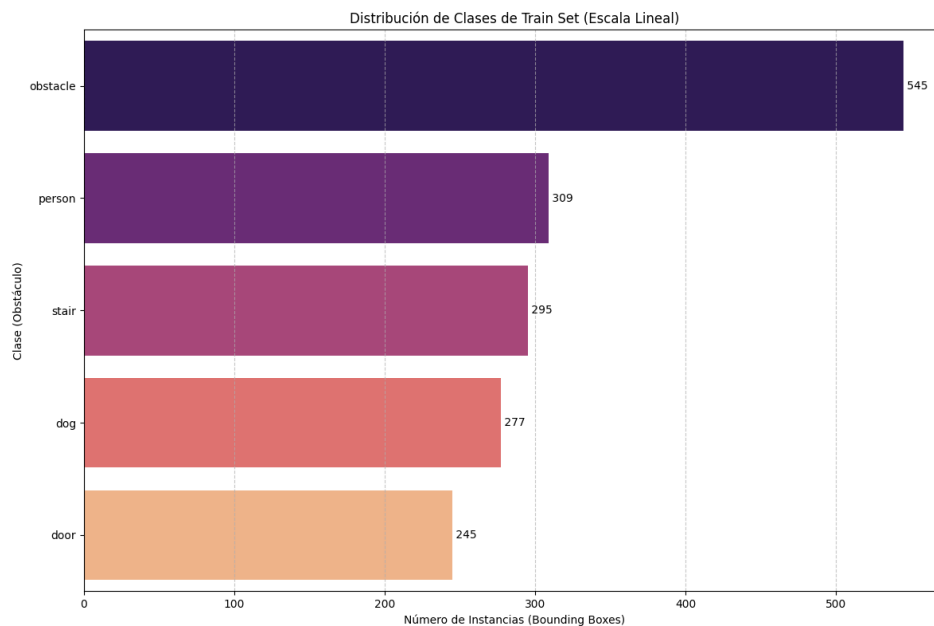


Figura 29. *Ratio Objetos/Imagen por Clase de Train Set*

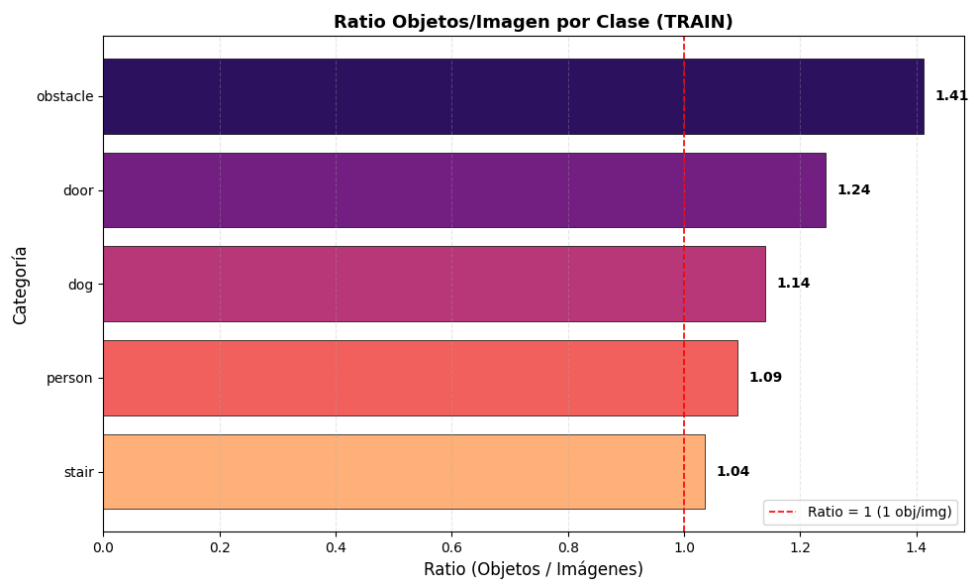
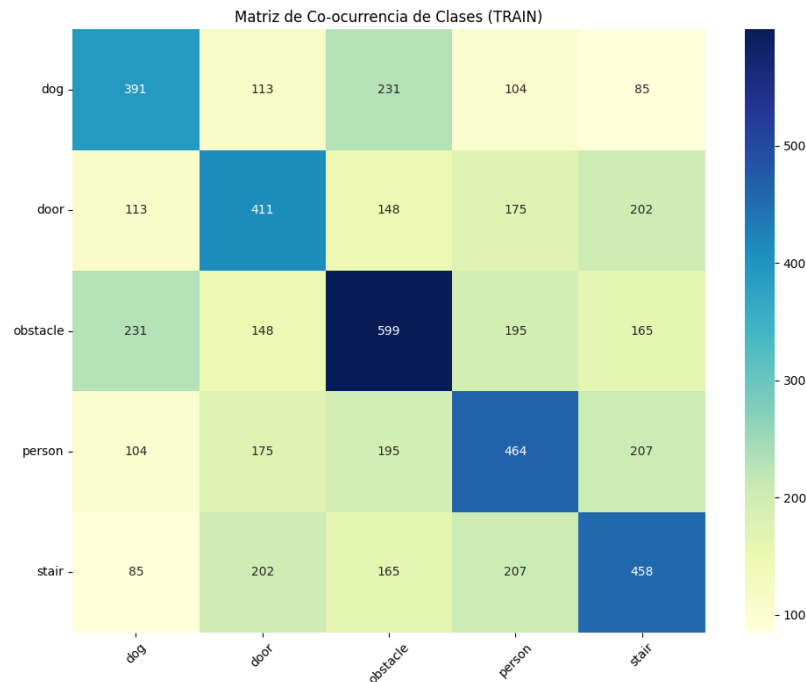


Figura 30. Matriz de Co-ocurrencia de Clases Train Set



A partir de estos resultados, se observa que la clase *obstacle* (545) duplica en frecuencia a *door* (245) y supera significativamente a *dog* (277), *stair* (295) y *person* (309). Por esta razón, se procede a estructurar un plan de aumentación de balanceo argumentado en las siguientes hipótesis:

- **Navegación Semántica y Crítica:** Las clases *door* (transición de espacios) y *stair* (límite/riesgo físico) son estructuralmente más importantes que un *obstacle* genérico. Un sesgo hacia *obstacle* podría causar que el LCMR intente atravesar una “puerta” o ignore el peligro de una “escalera” al clasificarlos erróneamente como objetos estáticos simples.
- **Seguridad Biológica:** Las clases *person* y *dog* requieren un Recall alto. Dado que el modelo se cuantizará a INT8, el margen de error se reduce; un dataset equilibrado asegura que los pesos de la red no se saturen con características de *obstacle*, permitiendo una discriminación fina de agentes móviles.
- **Compensación de Ratios:** El ratio de objetos por imagen es bajo (1.04 a 1.24 para las clases débiles). Esto significa que los modelos podrían ver pocos ejemplos por captura; el balanceo compensa esta baja densidad de información por frame.

El plan de aumentación de balanceo, también denominado *aumentación offline* por realizarse pre-entrenamiento, se basó en las reglas definidas en la Tabla 12. Se implementó una mezcla de aumentaciones geométricas, fotométricas y de composición para compensar aspectos como: cámara fija, reconocimiento de objetos desde múltiples ángulos, condiciones de luz y resolución baja de la cámara y diferentes escalas.

Tabla 12. Reglas de Aumentación Offline para Balanceo de Clases Train Set

Clase minoritaria	Aumento Objetivo	Técnicas	Lógica de Configuración
Dog	+97% (de 277 a 545)	Flip ⁷ Horizontal + Gaussian Noise	Invertir x y añadir ruido ($\sigma = 0.02$) para emular sensor OV5640 en noche.
Door	+122% (de 245 a 545)	Flip Horizontal + Perspective Warp ⁸	Invertir x y aplicar skew ⁹ leve ($\pm 5\%$) para simular aproximación diagonal del LCMR.
Stair	+85% (de 295 a 545)	Flip Horizontal + Crop/Zoom ¹⁰	Invertir x y hacer zoom ($1.1x$) para simular que el robot está más cerca del límite estructural.
Person	+76% (de 309 a 545)	Flip Horizontal + Blur/Sharpen ¹¹	Invertir x y aplicar desenfoque leve para simular movimiento relativo persona-LCMR.

La ejecución del pipeline de aumentación permitió expandir el set de entrenamiento de 874 a 1470 imágenes y de 1671 a 2744 anotaciones totales. Se alcanzó con éxito el objetivo de balanceo (*Target*) en todas las categorías propuestas. Uno de los mayores retos técnicos fue el control de la co-ocurrencia de clases, donde el aumento de una clase débil podría inflar desproporcionadamente a la clase fuerte. Para ello, se implementó una lógica de freno por cascada que rechazó la generación de muestras cuando el crecimiento colateral superaba los umbrales definidos. Como resultado, se registraron 166 rechazos, lo que permitió que la clase *obstacle* solo creciera en 16 instancias (+2.9%), como se constata en la Figura 31 con la matriz

⁷ Flip Horizontal [Volteo Horizontal], inversión de la imagen sobre su eje vertical para crear un efecto espejo.

⁸ Perspective Warp [Deformación de perspectiva], ajuste de los planos de una imagen para corregir o simular profundidad y ángulos de visión.

⁹ Skew [Sesgar], Inclinación de la imagen en dirección lateral o vertical, distorsionando sus ángulos originales.

¹⁰ Crop/Zoom [Recortar/Zoom], El recorte elimina áreas externas de la imagen; el zoom aumenta el tamaño visual de una sección específica.

¹¹ Blur/Sharpen [Desenfocar/Enfocar], suavizar los bordes para reducir el detalle o aumentar el contraste en los bordes para mejorar la nitidez.

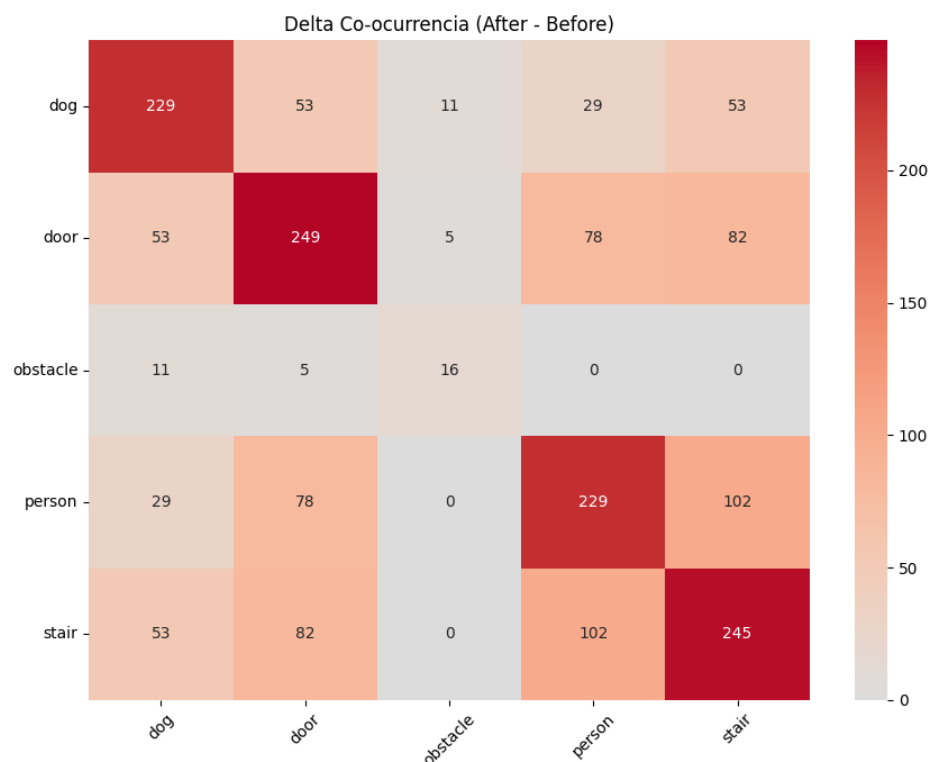
delta de co-ocurrencia (antes – después), manteniendo un balance efectivo sin un inflado descontrolado de la clase dominante.

La Figura 31 también comprueba que se lograron aumentos significativos en pares de clases fundamentales para la navegación del LCMR, como:

- **person - stair (+102) y door - stair (+82):** El modelo ahora tiene muchas más muestras de humanos cerca de escaleras y puertas en el set de entrenamiento. Esto es vital para que el LCMR aprenda la semántica de "agente biológico en zona de riesgo".
- **door - person (+78):** Refuerza el aprendizaje de personas en puntos de transición de espacios.

Los valores bajos en combinaciones como **dog - obstacle (+11)** en comparación con el aumento total de **dog (+229)** muestran que la mayoría de los nuevos “perros” inyectados al dataset provienen de imágenes "puras" o de baja co-ocurrencia con obstáculos.

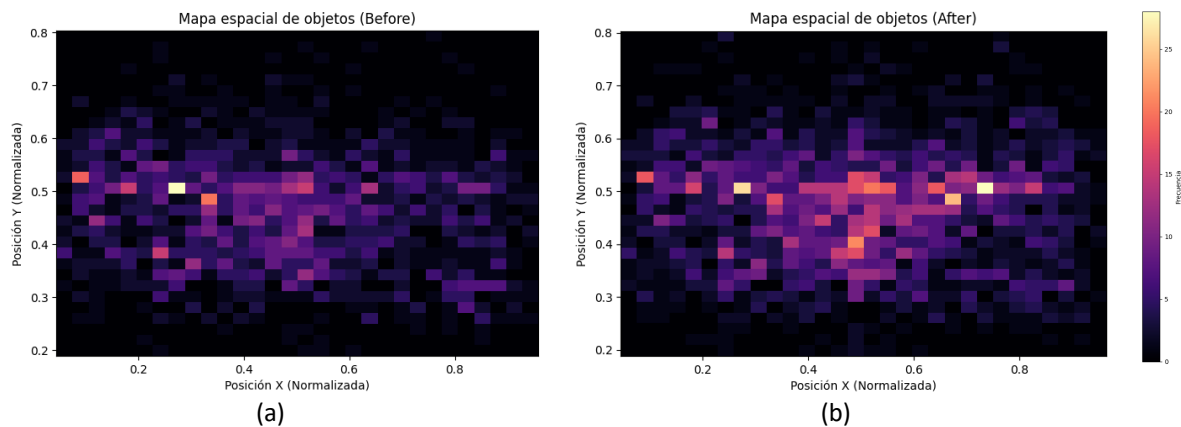
Figura 31. Matriz Delta de Co-ocurrencia de Clases Train Set (After-Before)



Por otro lado, la comparación de mapas espaciales de objetos antes y después de la aumentación (Figura 32), confirma que el proceso realizado respetó la perspectiva original, manteniendo los objetos alineados con el horizonte visual y manteniendo la concentración de

centroides en la franja horizontal central. En el mapa espacial original (antes), se observaba un área de alta frecuencia pronunciada hacia el lado izquierdo. En el mapa después, la distribución se equilibró y se observa mucho más densa a lo largo de todo el eje 'x'. Este cambio es producto del volteo horizontal (*Horizontal Flip*) aplicado, eliminando sesgos compositivos originales y entregando más variabilidad para detectar objetos sin importar su ubicación lateral en el frame. De igual manera, se mantuvo la “zona de exclusión” (negra) en los extremos superior ($Y > 0.8$) e inferior ($Y < 0.2$) certificando que las aumentaciones no alteraron la lógica semántica de la ubicación de los objetos y del plano de tierra indispensable para el LCMR.

Figura 32. Mapa Espacial de Objetos Antes y Después de Aumentación



Nota: (a) Mapa Espacial de Objetos Antes de la Aumentación. (b) Mapa Espacial de Objetos Después de la Aumentación.

4.8. Síntesis

[Como síntesis de la fase de diseño e ingeniería de datos, el conjunto resultante presenta una caracterización estadística que condicionará de forma directa la estrategia de entrenamiento de los modelos. En términos de Análisis de Distribución Post-Procesamiento, se obtuvo una asimetría marcada donde la clase *obstacle* domina el dataset con un 58.7% de las instancias, mientras que las categorías *person* y *door* representan apenas el ~10% cada una. Si bien el dataset se considera limpio, consistente desde una perspectiva geométrica y con un volumen aceptable para la aplicación de Transfer Learning, este desbalance no corregido mediante el

aumento uniforme (para prevenir sesgos inducidos) introduce un riesgo de sesgo algorítmico que debe ser abordado mediante las siguientes acciones durante para la Fase de Modelado:

- **Implementación de Class Weights:** Es imperativo calcular y aplicar pesos específicos a la función de pérdida (Loss Function) durante el entrenamiento para penalizar con mayor rigor los errores en las clases person y door.
- **Métrica de Evaluación Ponderada:** Se monitorizará el mAP (mean Average Precision) por clase como indicador de éxito, evitando depender del accuracy global que podría estar inflado por la clase mayoritaria.
- **Configuración de Anchors:** Las cajas de anclaje se calibrarán ignorando las dimensiones de los objetos minúsculos eliminados en la purga lógica.
- **Estrategia de Transfer Learning:** Dado que el volumen total tras el aumento se mantiene por debajo de las 2,000 imágenes, se utilizarán pesos pre-entrenados de diversos modelos para asegurar la convergencia, realizando un congelamiento inicial de las capas base

]